



南京大學
NANJING UNIVERSITY

二、STM32软硬件基础

主讲教师：闫浩
机器人与自动化学院



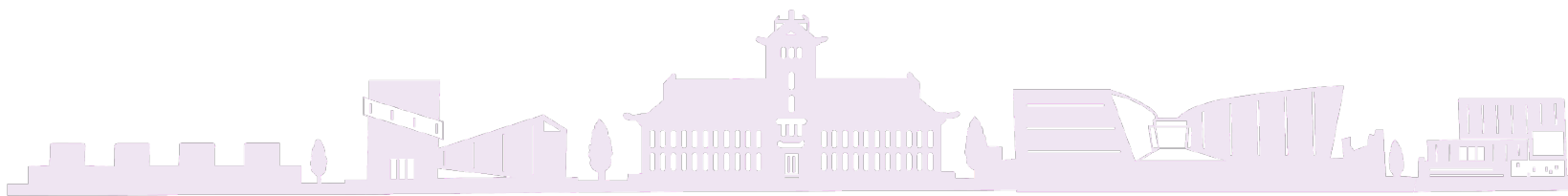
目录

CONTENTS

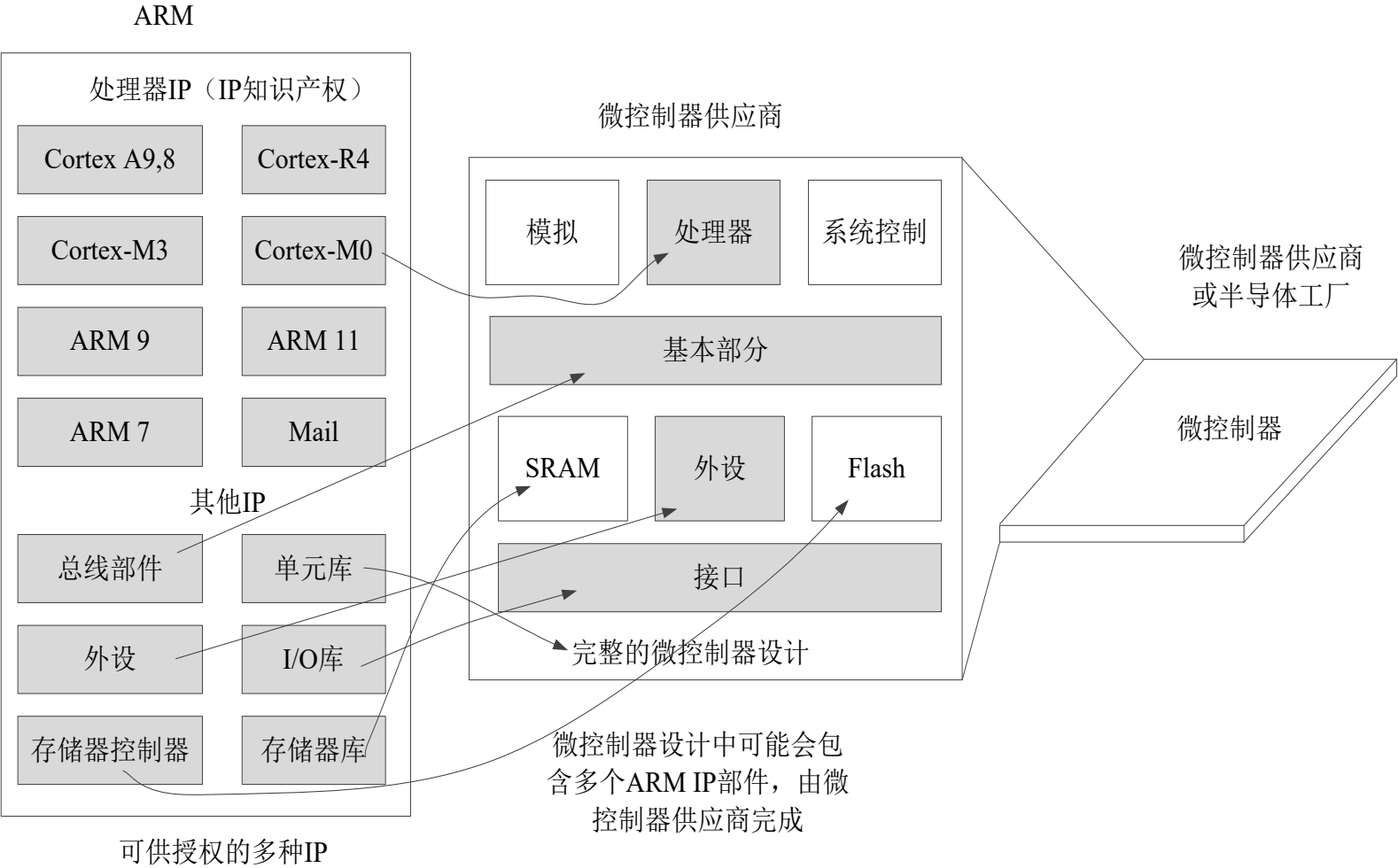
01 STM32概述

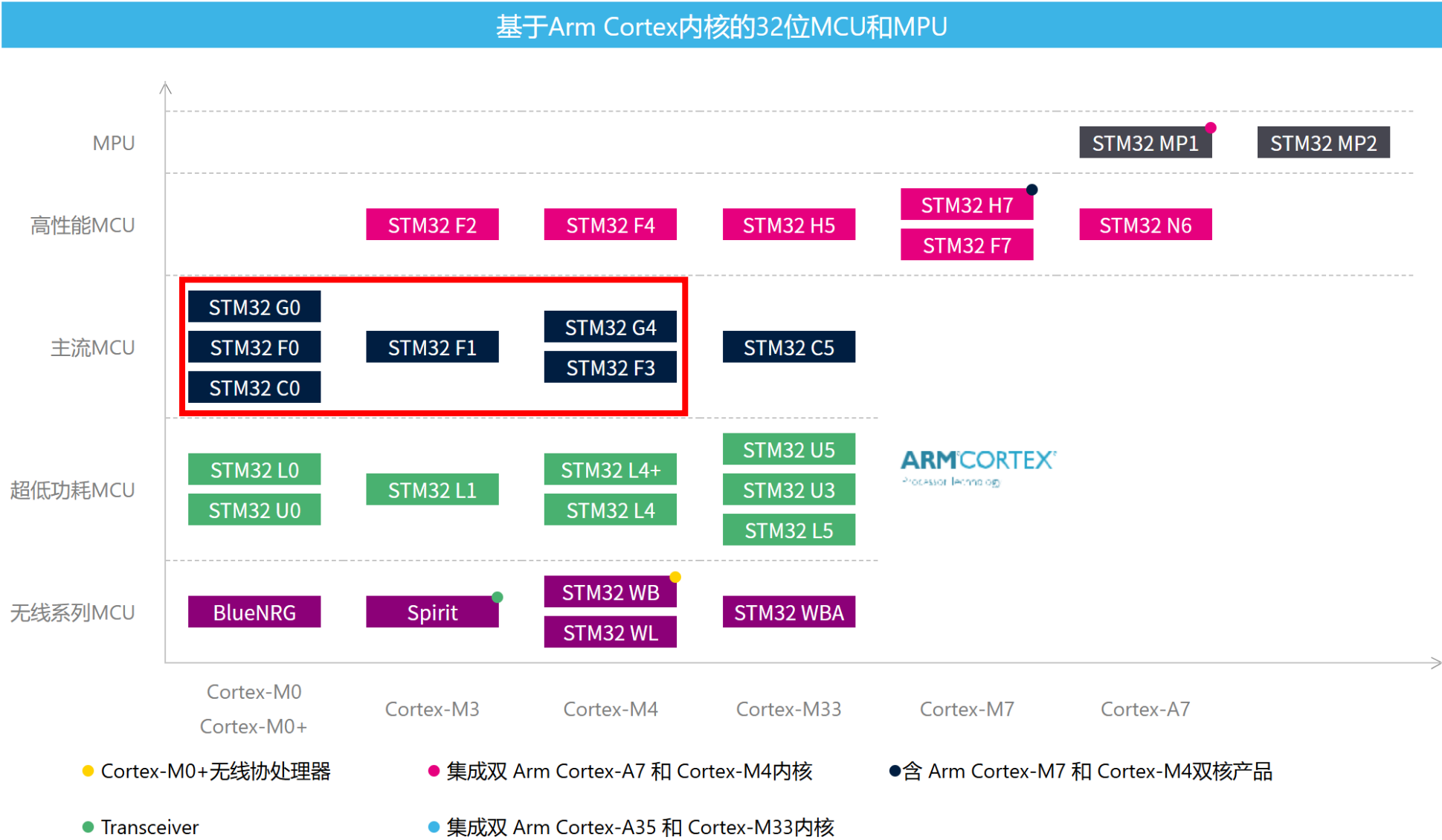
02 最小系统设计

03 开发环境搭建



01 STM32概述





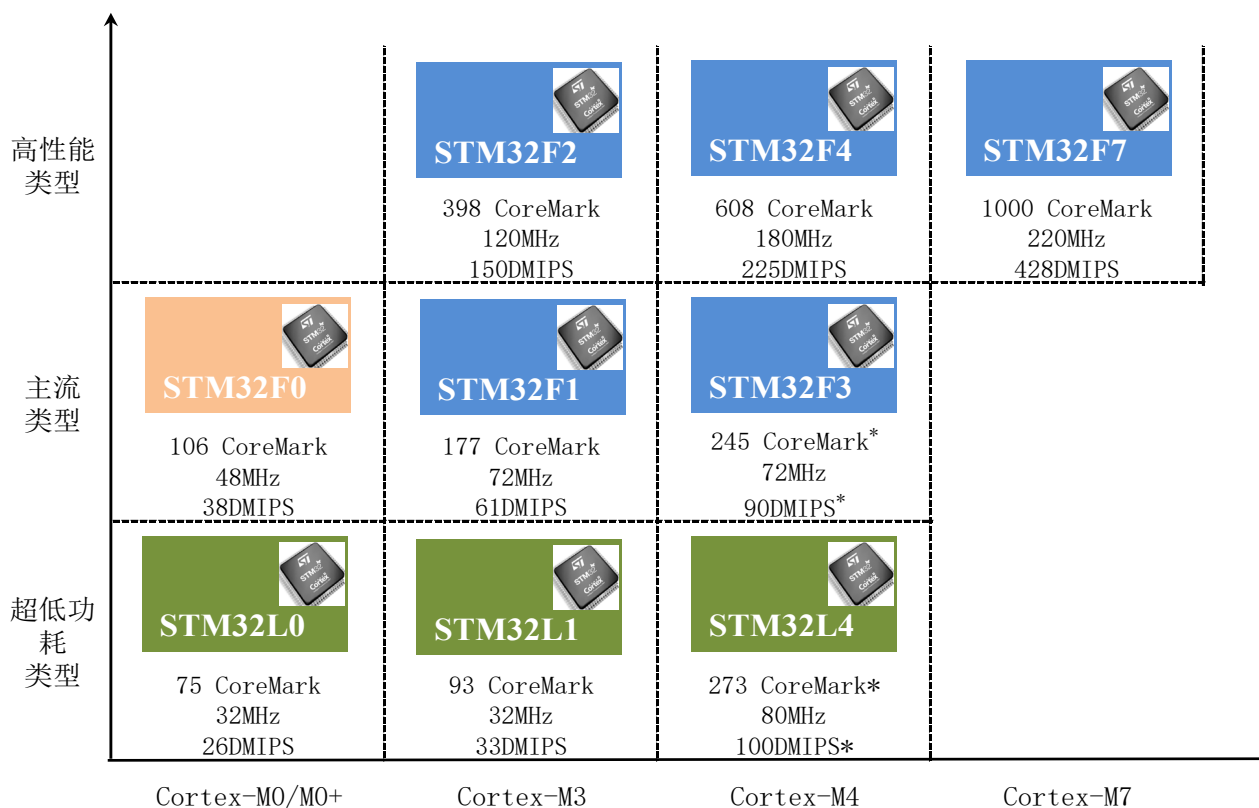
STM32是**意法半导体 (STMicroelectronics)** 较早推向市场的基于Cortex-M内核的微处理器系列产品，该系列产品具有**成本低、功耗优、性能高、功能多**等优势，并且以系列化方式推出，方便用户选型，在市场上获得了广泛好评。

STM32 目前常用的有STM32F103 ~ 107系列，简称“**1系列**”，最近又推出了高端系列STM32F4xx系列，简称“**4系列**”。前者基于Cortex-M3内核，后者基于Cortex-M4内核。STM32F4xx系列在以下诸多方面做了优化：

- (1) 增加了浮点运算
- (2) DSP处理
- (3) 存储空间更大，高达1M字节以上
- (4) 运算速度更高，以168MHz高速运行时可达到210DMIPS的处理能力
- (5) 更高级的外设，新增外设，例如，照相机接口、加密处理器、USB 高速OTG 接口等，提高性能，更快的通信接口，更高的采样率，带FIFO的DMA控制器

STM32系列微控制器不适合的应用有：程序代码大于1MB的应用，基于Linux或Android的应用，基于高清或超高清的视频应用等。

STM32系列微控制器的产品线包括**高性能类型**、**主流类型**和**超低功耗类型**三大类，分别面向不同的应用，其具体产品系列如下图所示。



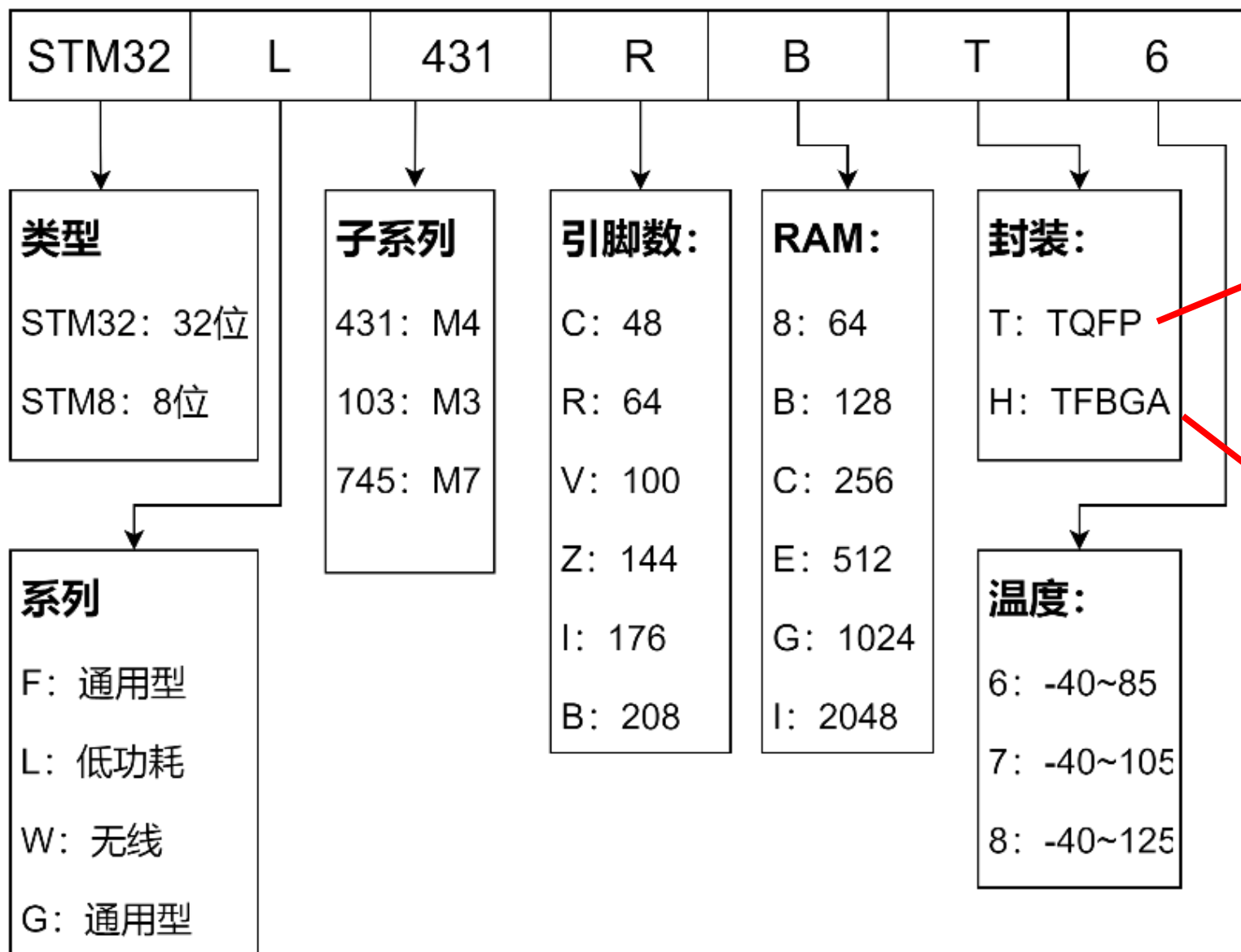
STM32F1系列微控制器 (**Cortex-M3内核**) 包含以下5个产品线, 相互兼容。

- 1) STM32F100, 超值型, 24MHz CPU, 具有电机控制和CEC功能。
- 2) STM32F101, 基本型, 36MHz CPU, 具有高达1MB的Flash。
- 3) STM32F102, USB基本型, 48MHz CPU, 具备USBFS。
- 4) **STM32F103, 增强型, 72MHz CPU, 具有高达1MB的Flash、电机控制、USB和CAN。**
- 5) STM32F105/107, 互联型, 72MHz CPU, 具有以太网MAC、CAN和USB2.0 OTG。

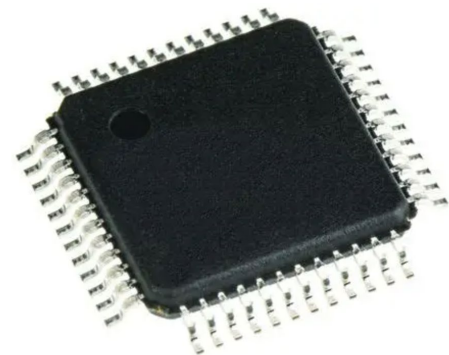
STM32F4系列微控制器 (**Cortex-M4内核**) :

- 1) STM32F401, 84MHz CPU/105DMIPS, 尺寸最小、成本最低的解决方案, 具有卓越的功耗效率。
- 2) STM32F410, 100MHz CPU/125DMIPS, 采用新型智能DMA, 优化了数据批处理的功耗 (采用批采集模式的动态效率系列), 配备的随机数发生器、低功耗定时器和DAC, 为卓越的功率效率性能设立了新的里程碑。
- 3) STM32F411, 100MHz CPU/125DMIPS, 具有卓越的功率效率、更大的SRAM和新型智能DMA, 优化了数据批处理的功耗。
- 4) **STM32F407/417, 168MHz CPU/210DMIPS, 高达1MB的Flash闪存, 增加了以太网MAC和照相机接口。**

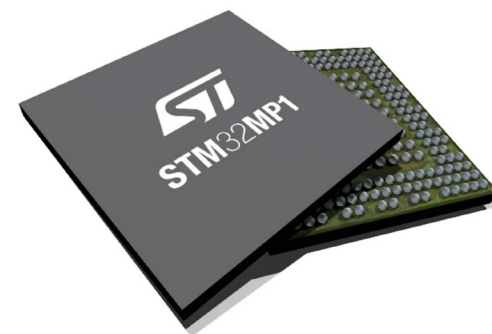
STM32微控制器的命名规则



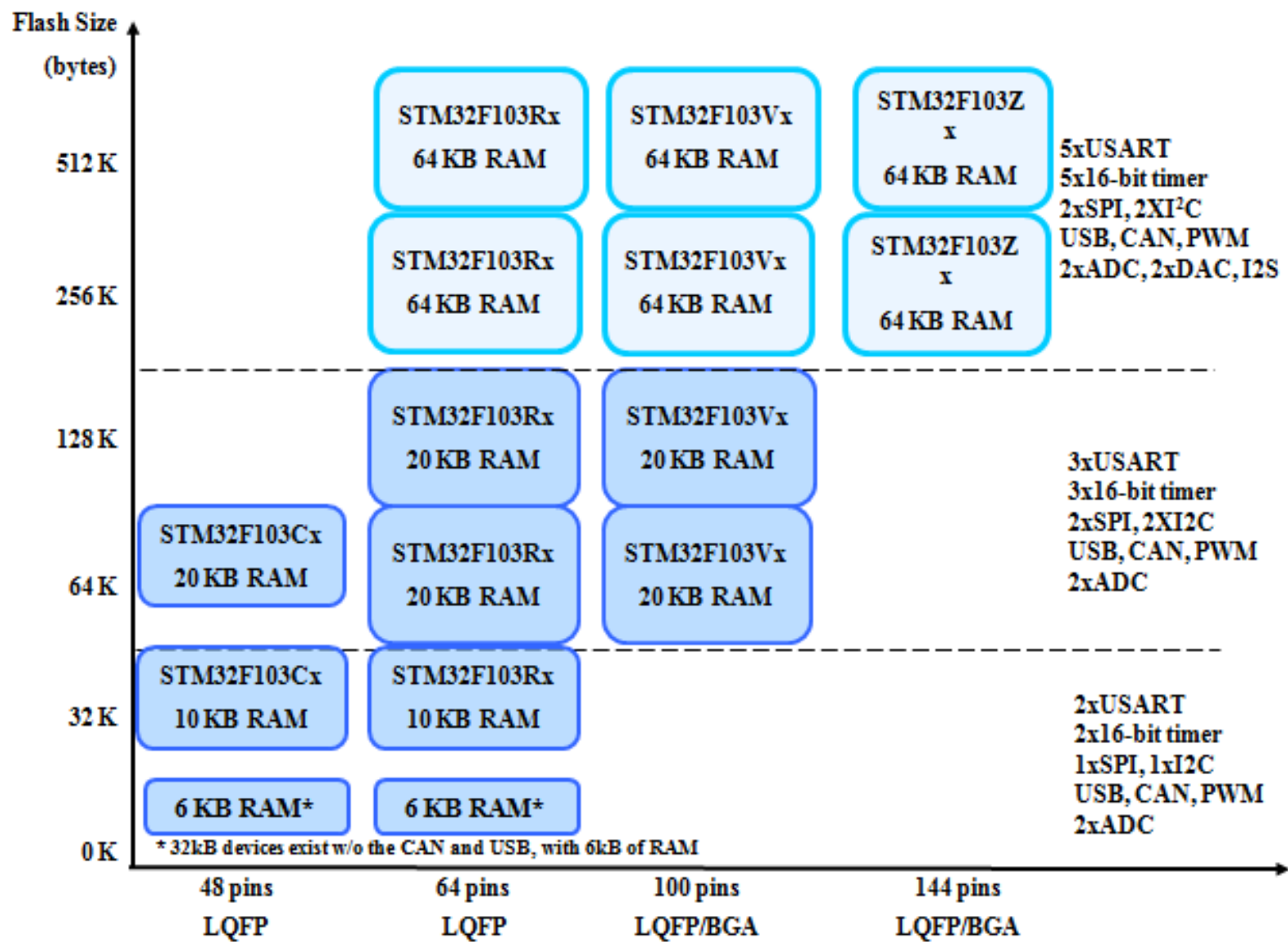
一种薄型四方扁平封装



薄型细间距球栅阵列封装



STM32F103xx闪存容量、封装及型号对应关系



买电脑时，关心哪些参数？？？



- CPU主频
- 内存
- 硬盘
- 显卡
- 电源功率
- . . .

STM32 跟其他单片机一样，是一个单片计算机或单片微控制器，**所谓单片就是在一个芯片上集成了计算机或微控制器该有的基本功能部件。这些功能部件通过总线连在一起。**

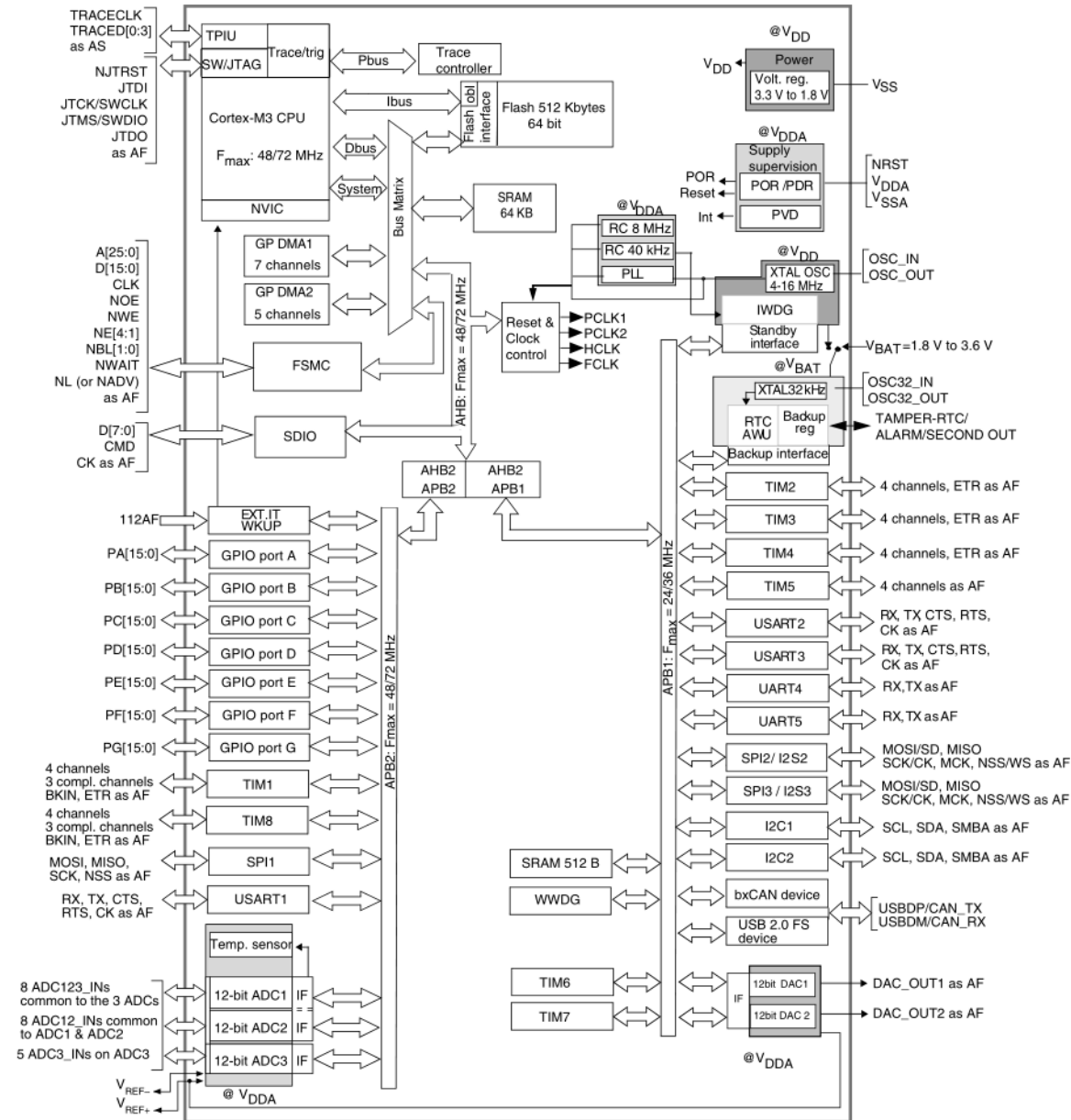
这些功能部件主要包括：

- Cortex-M 内核
- 总线
- 系统时钟发生器
- 复位电路
- 程序存储器
- 数据存储器
- 中断控制
- 调试接口以及各种功能部件（外设）

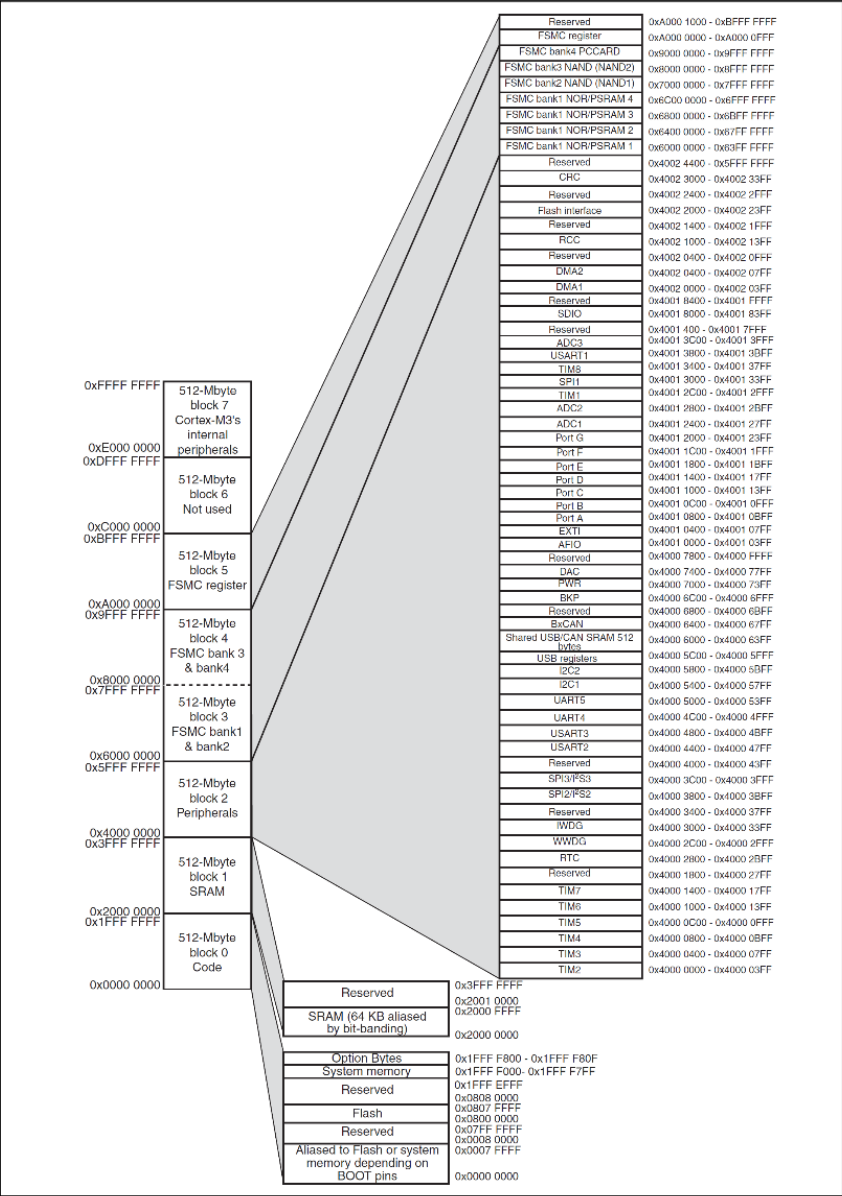
STM32F10x系列单片机基于ARM Cortex-M3内核，其结构特点为：

- 1) 一个**主晶振**可以驱动整个系统，低成本的4 ~ 16MHz晶振即可驱动CPU、USB和其他所有外设。
- 2) **内嵌**出厂前调校好的8MHz RC振荡器，可以作为低成本主时钟源。
- 3) 内嵌电源监视器，减少对外部器件的要求，提供上电复位、低电压检测、掉电检测。
- 4) **GPIO**：最大翻转频率为18MHz。
- 5) **PWM定时器**：可以接收最大72MHz时钟输入。
- 6) **USART**：传输速率可达4.5Mbit/s。
- 7) **ADC**：12位，转换时间最快为1 μ s。
- 8) **DAC**：提供2个通道，12位。
- 9) **SPI**：传输速率可达18Mbit/s，支持主模式和从模式。
- 10) **I2C**：工作频率可达400kHz。
- 11) **I2S**：采样频率可选范围为8 ~ 48kHz。
- 12) 自带时钟的**看门狗定时器**。
- 13) **USB**：传输速率可达12Mbit/s。
- 14) **SDIO**：传输速率为48MHz。

STM32F103ZET6的内部架构



STM32F103ZET6的存储器映像



程序运行的本质是什么???

程序存储器

数据存储器

寄存器

输入 / 输出端口

STM32的时钟树

STM32系列微控制器中，有4个主要时钟源，分别是：

1 高速内部时钟

(High Speed Internal. HSI)

2 高速外部时钟

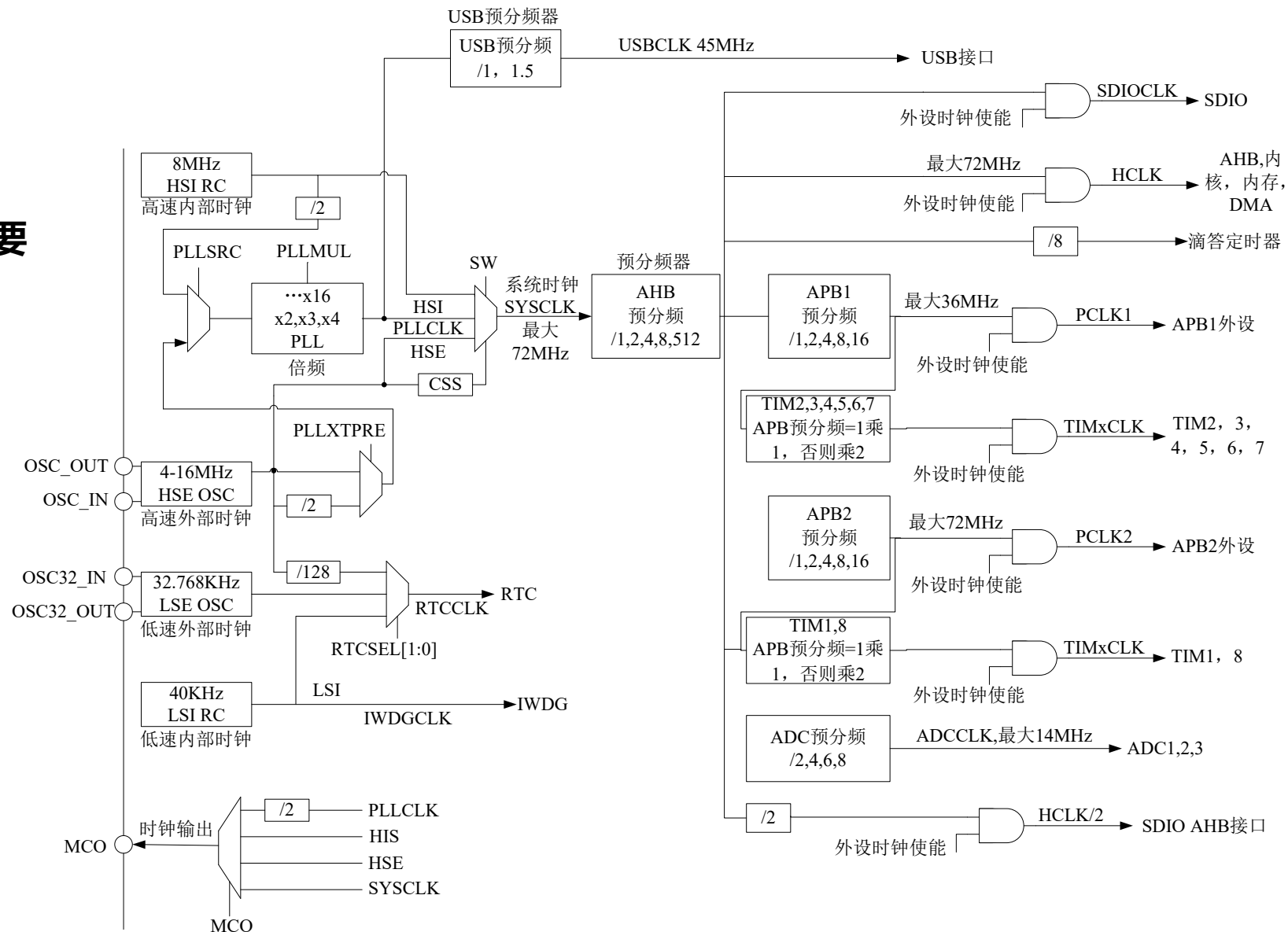
(High Speed External, HSE)

3 低速内部时钟

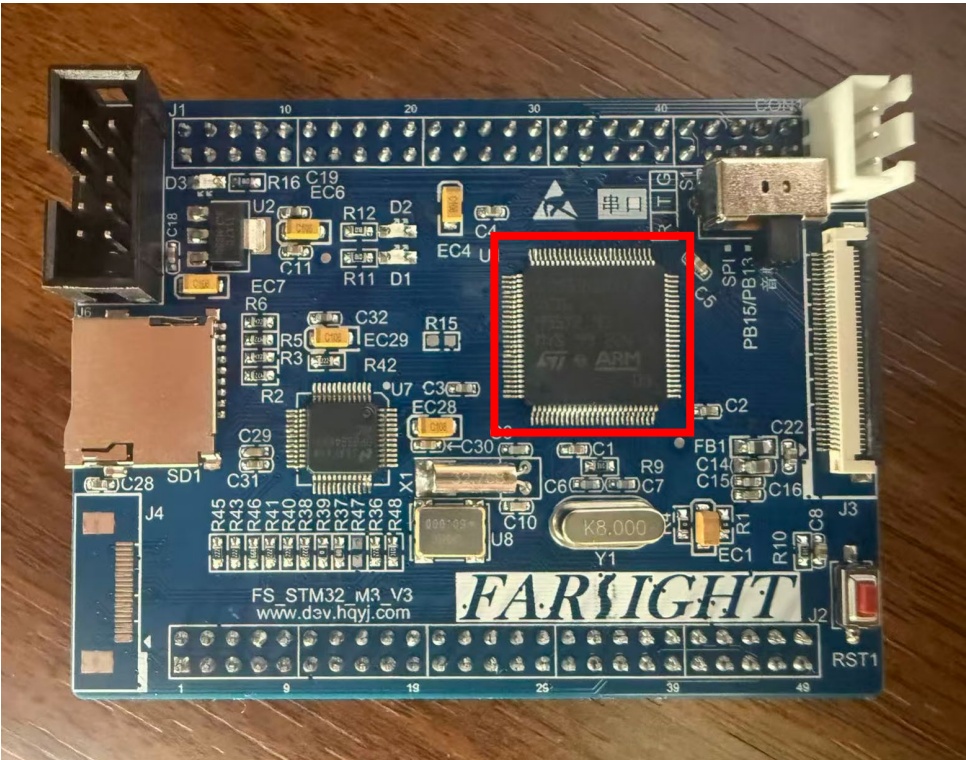
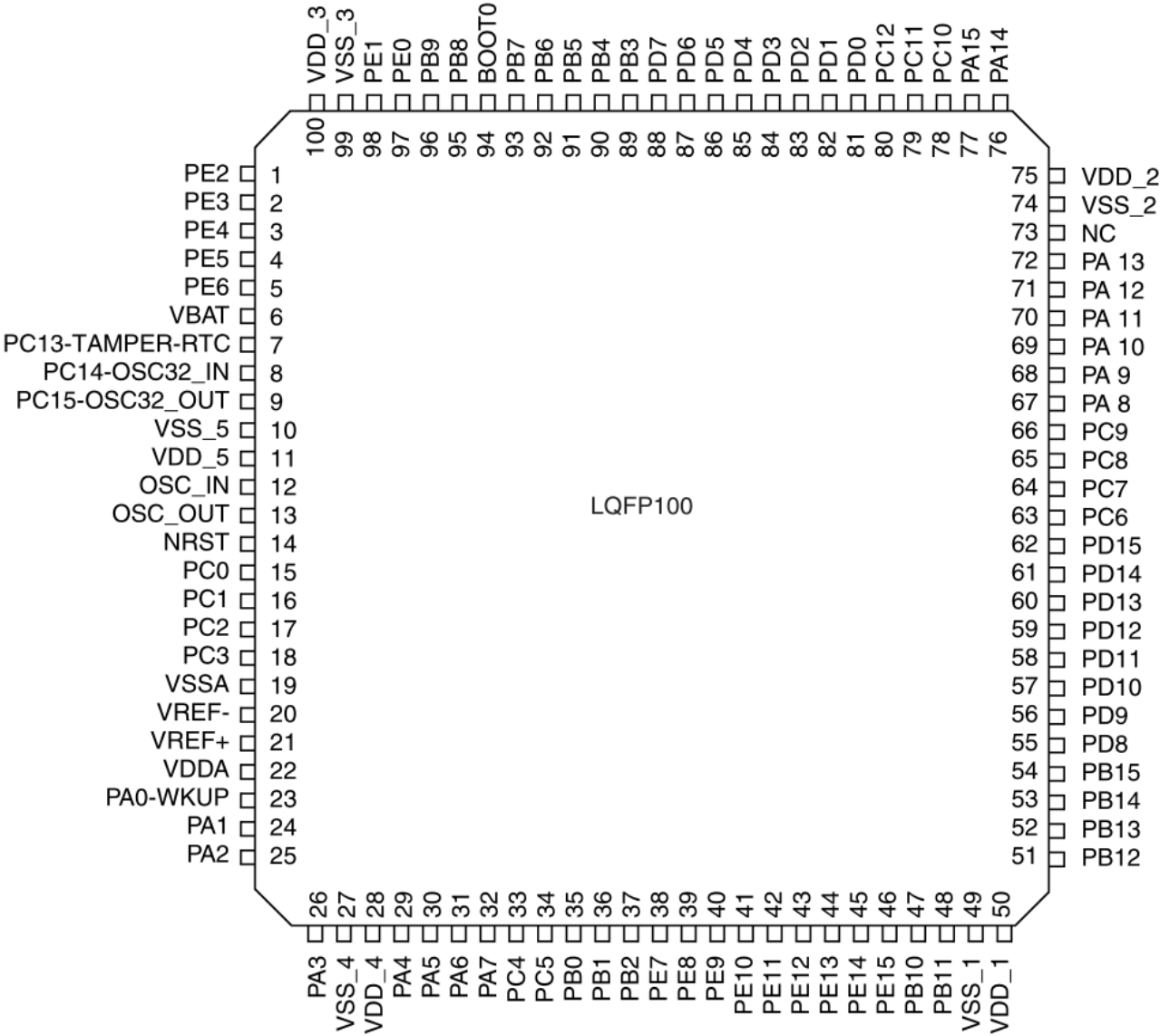
(Low Speed Internal. LSI)

4 低速外部时钟

(Low Speed External, LSE)



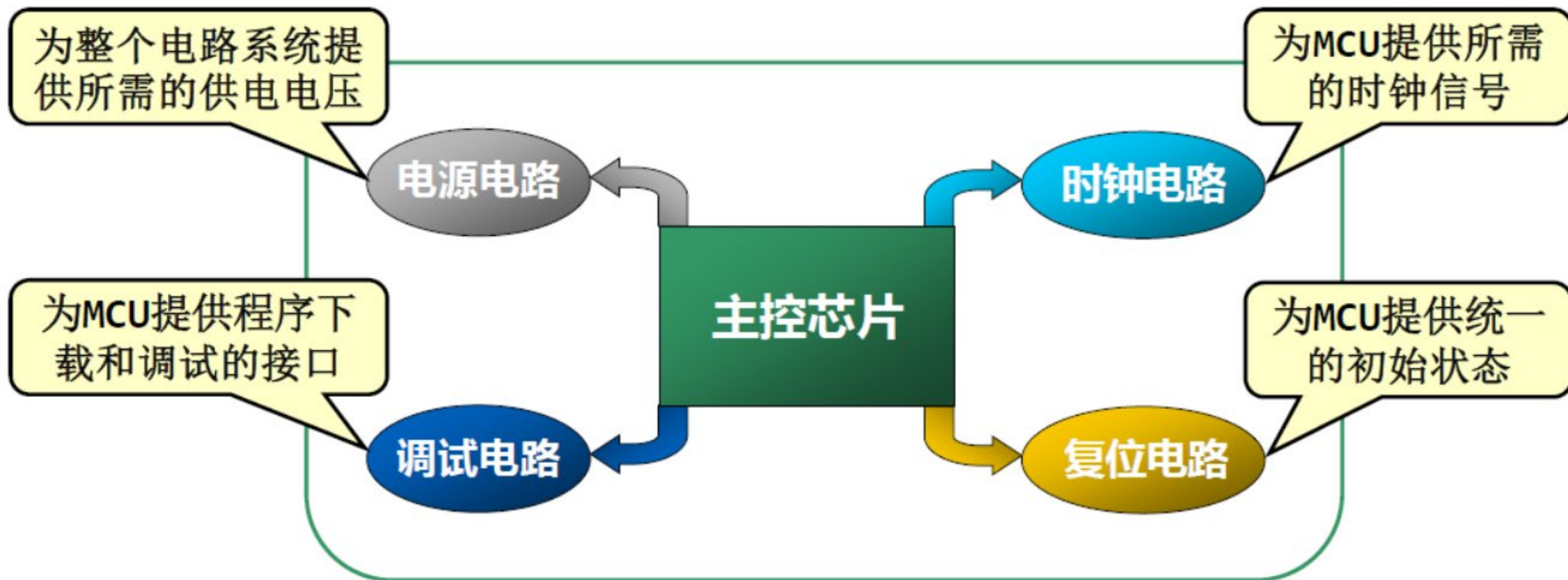
STM32F103VET6的引脚

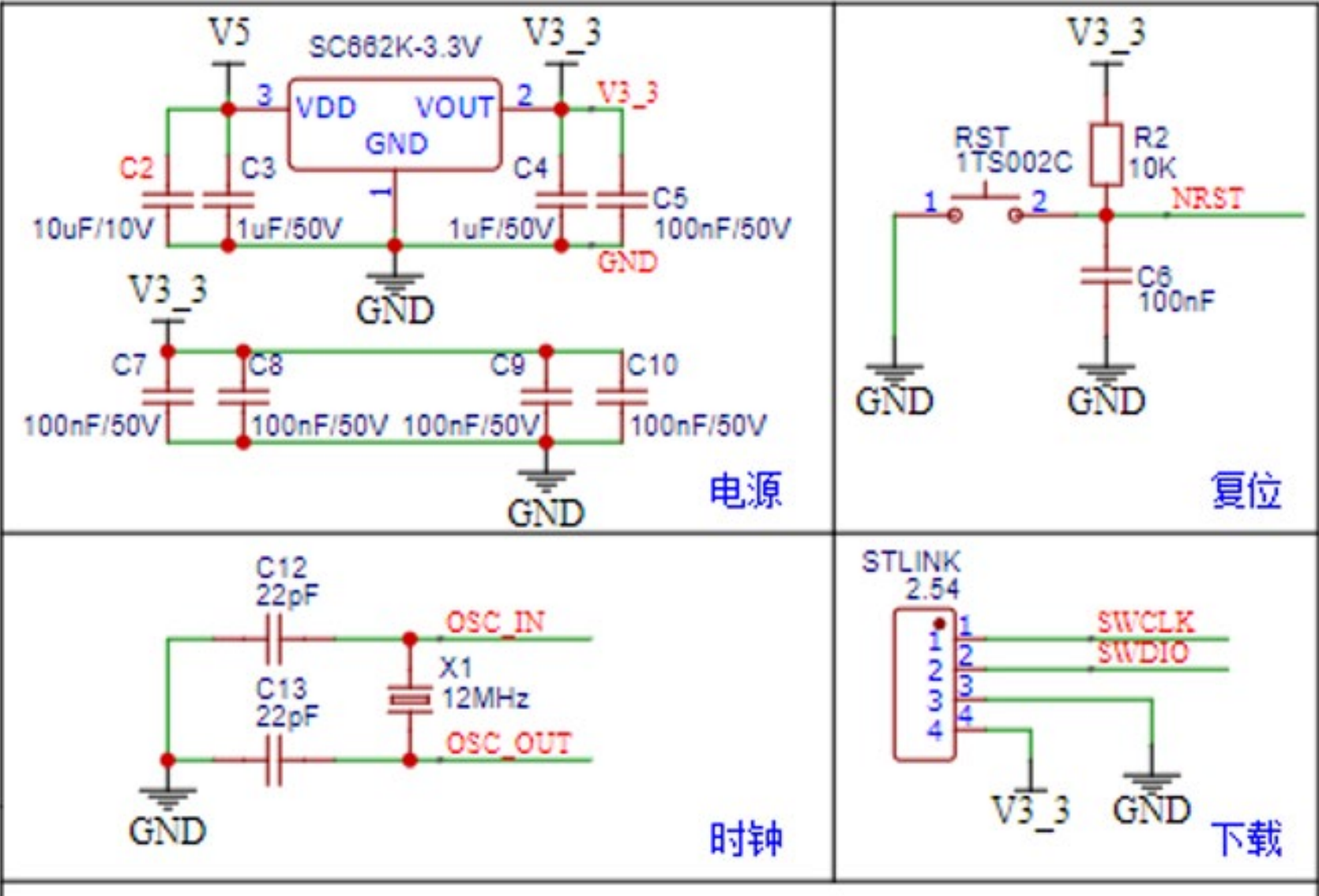


02 最小系统设计

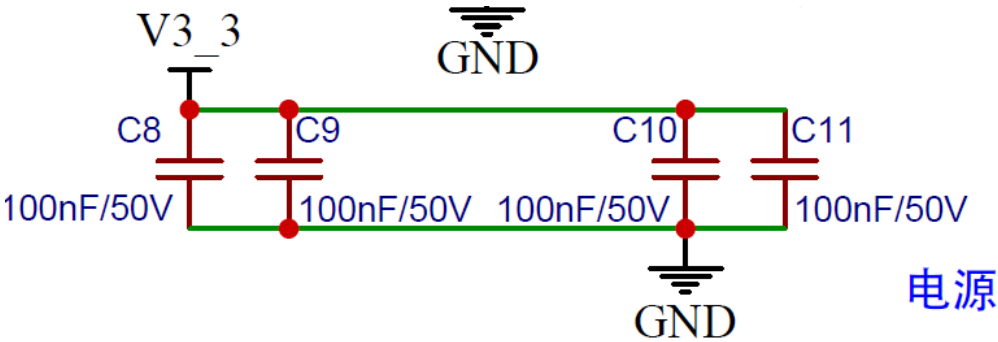
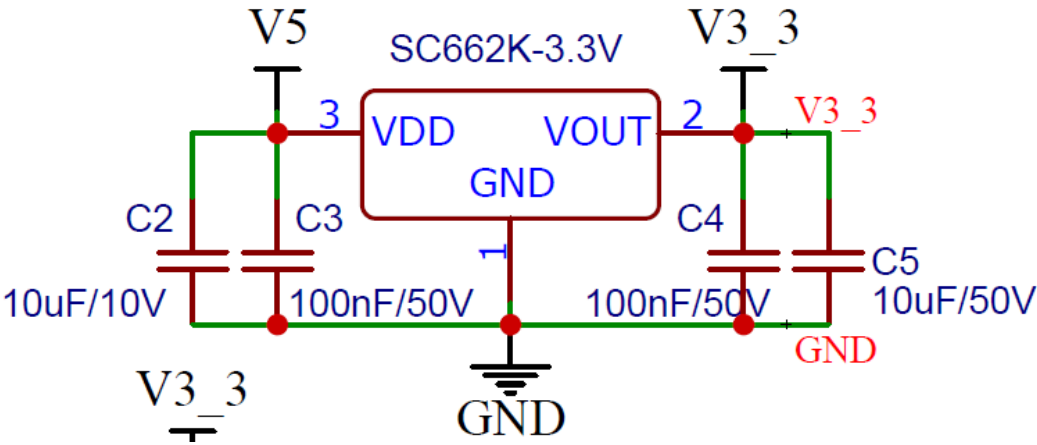
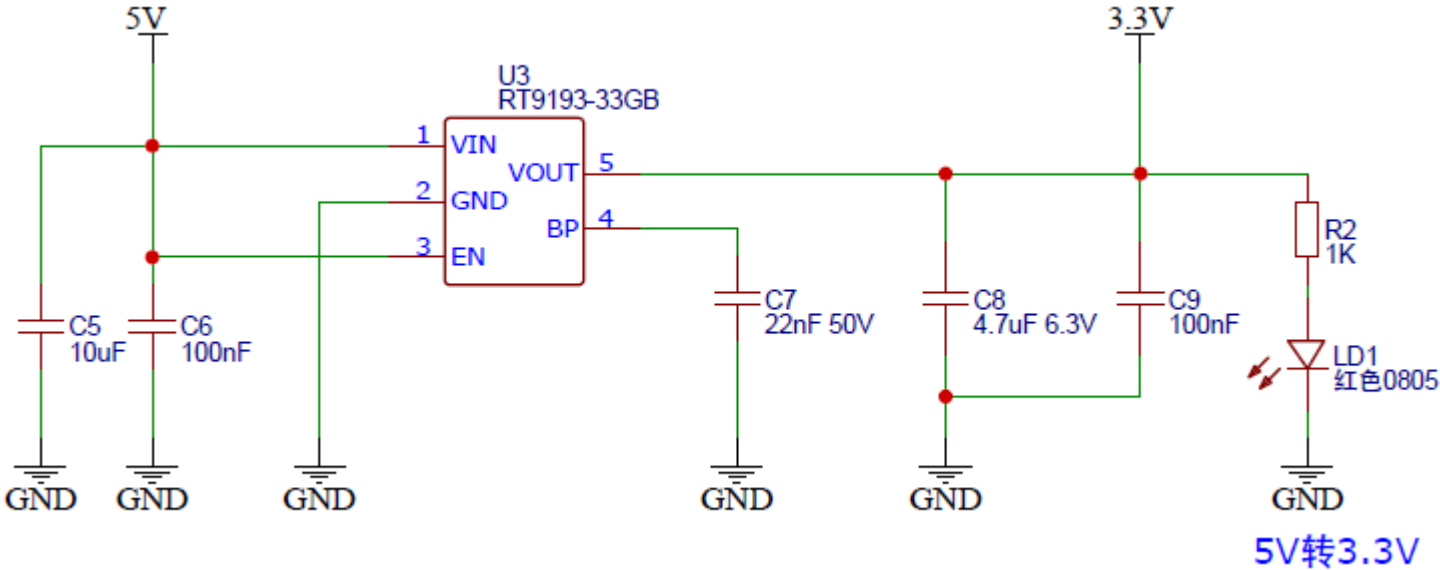
什么是最小系统？？？

最小系统是指能够让STM32单片机正常工作的包含最少元器件的系统。





电源



时钟

时钟源

STM32微控制器的四个时钟源

LSI



内部低速时钟。由芯片内部的RC振荡器提供，默认频率为32KHz

HSI



内部高速时钟。由芯片内部的RC振荡器提供，默认频率为16MHz

HSE



外部高速时钟。通过在OSC_IN和OSC_OUT引脚接入晶振实现，频率范围为4MHz~26MHz。也可以直接接入外部时钟信号，频率范围为1MHz~50MHz

LSE

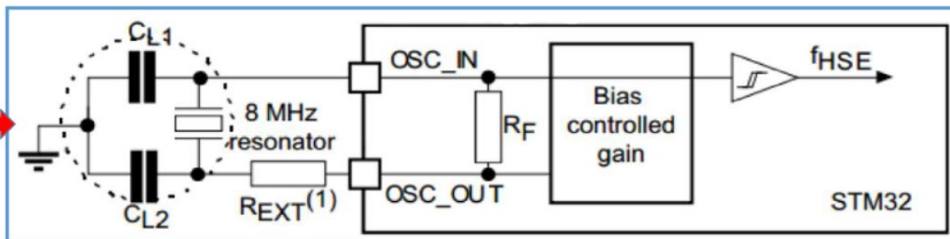


外部低速时钟。通过在OSC32_IN和OSC32_OUT引脚接入32.768KHz的晶振实现

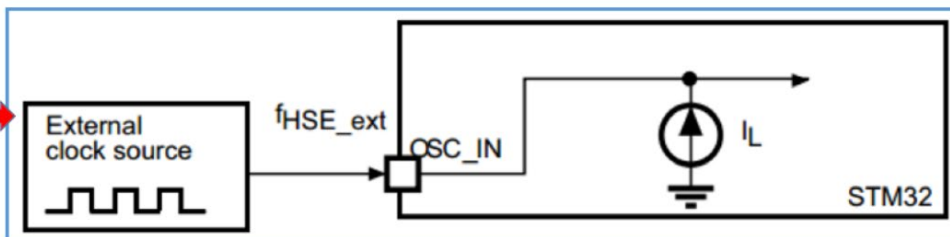
时钟电路

时钟电路的两种设计方法

振荡模式

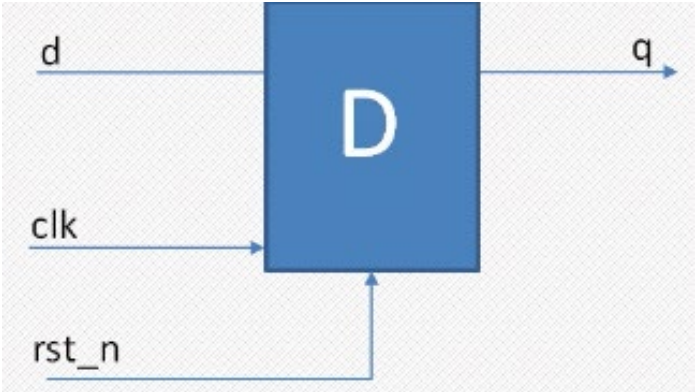


旁路模式

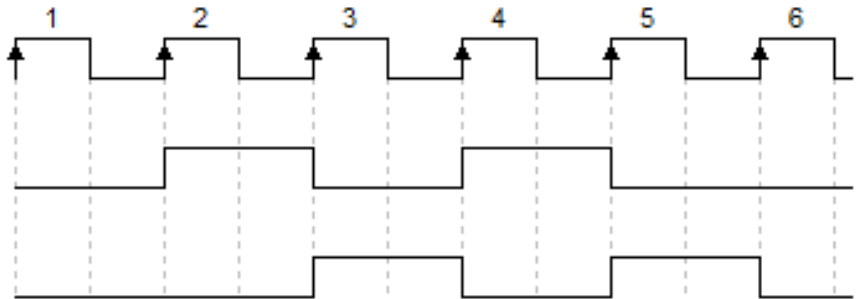


时钟提供时序，微控制器根据时序运行。

时钟



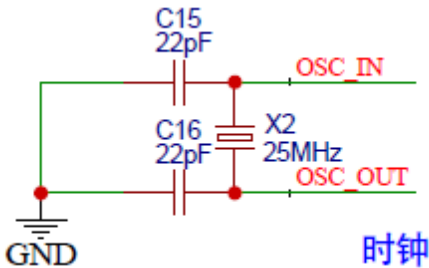
clk
d
q



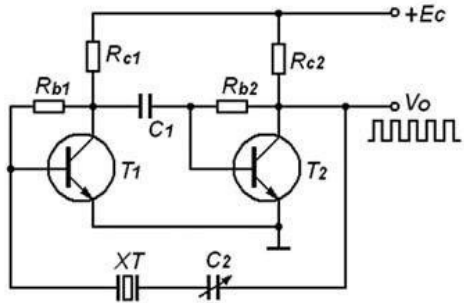
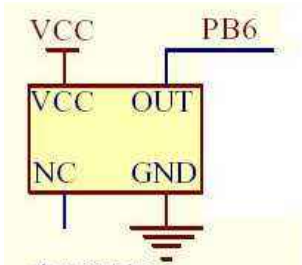
无源晶振：2个引脚；内部无电源，需CPU内部振荡器才能产生时钟信号；便宜



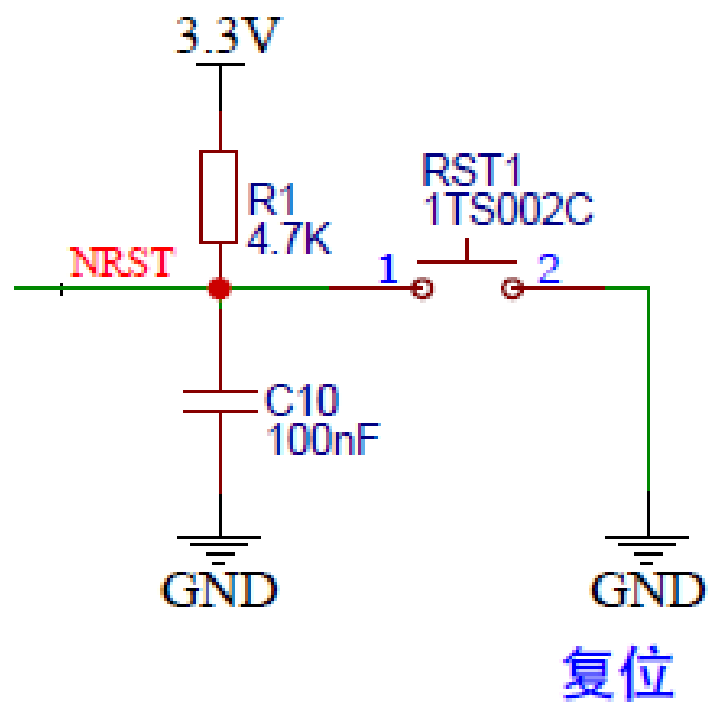
有源晶振：4个引脚；内部有震荡电路，通电直接输出时钟信号；稍贵。



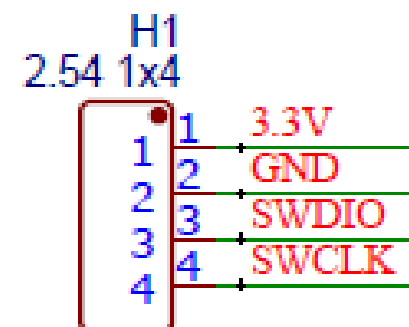
不起振：万用表测OSC IN和OSC OUT两端电压，低于 $\frac{1}{2} V_{CC}$ 。
程序下载失败！



复位



调试

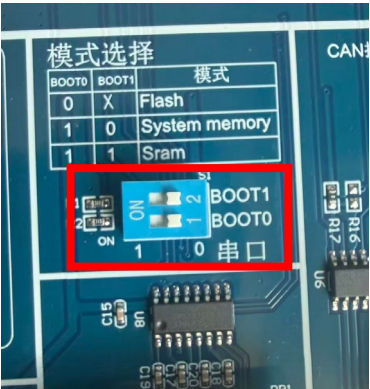
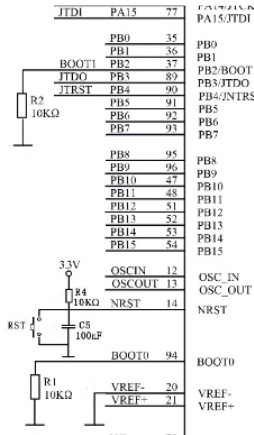


下载

启动配置引脚

在STM32F103VET6中，可以通过**BOOT [1:0]** 引脚选择3种不同的启动模式。STM32F103VET6的启动配置如下表所示。

启动模式选择引脚		启动模式	说明
BOOT1	BOOT0		
X	0	主闪存存储器	主闪存存储器被选为启动区域
0	1	系统存储器	系统存储器被选为启动区域
1	1	内置SRAM	内置SRAM被选为启动区域

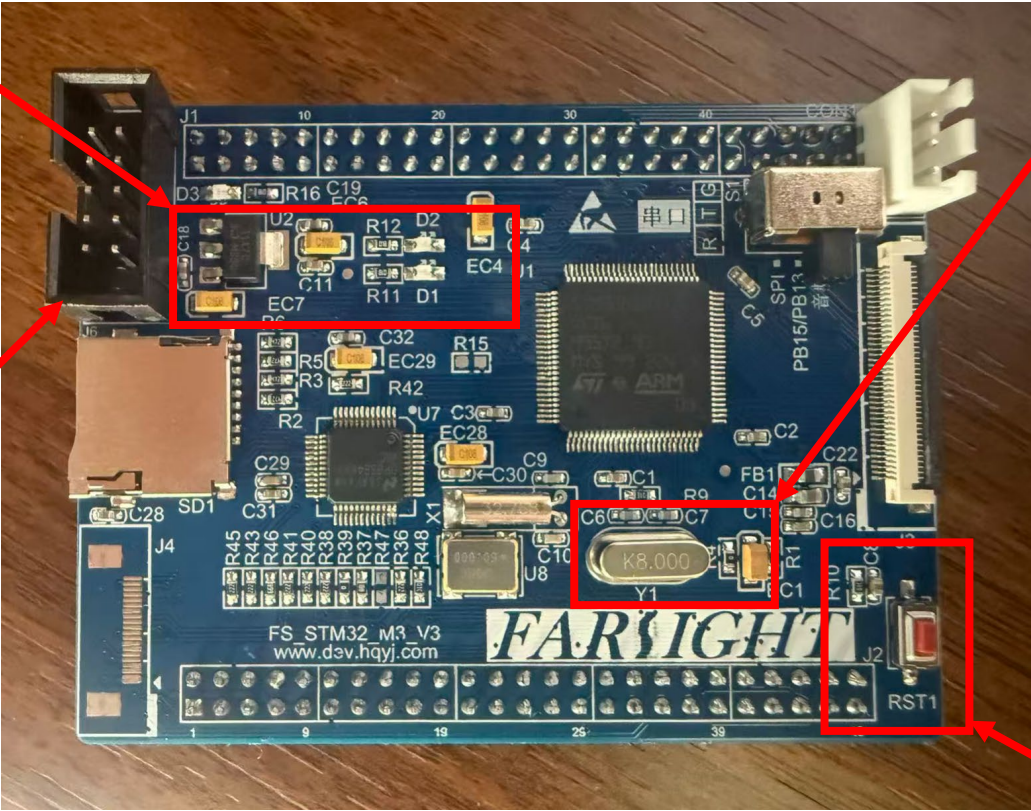


电源电路

时钟电路

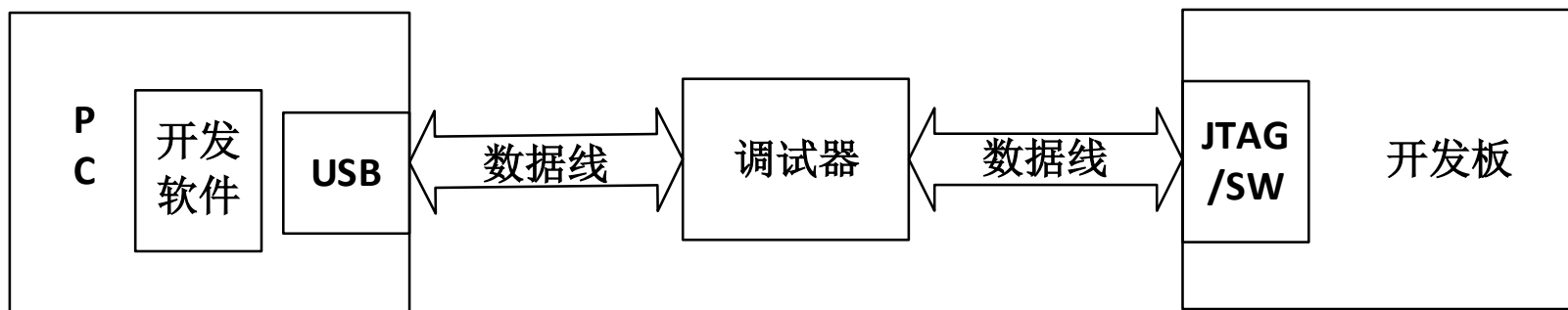
调试电路

复位电路



03 开发环境搭建

进行STM32F系列微控制器的程序开发需要搭建一个交叉开发环境，其中包括计算机、开发软件、调试器、开发板或自己设计的电路板（包括STM32F系列微控制器）。



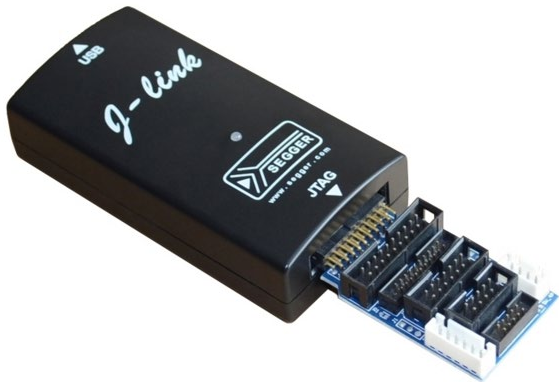
程序交叉开发环境搭建示意图

- 1、开发板；
- 2、软件集成开发工具；
- 3、调试工具；
- 4、其他：RTOS、开源协议栈。

调试器

(1) JLINK

J-Link 是SEGGER 公司为支持仿真ARM 内核芯片推出的JTAG 仿真器。是通用的开发工具，可以用于KEIL， IAR， ADS 等平台 速度，效率，功能均比ULINK强。需要安装驱动。



(2) ULINK

ULINK是KEIL公司开发的仿真器，专用于KEIL MDK平台。在KEIL MDK平台下无需驱动，可直接使用。



调试器

(3) STlink

STLink 是ST 公司为STM8和STM32系列MCU设计的调试器。需要安装驱动。



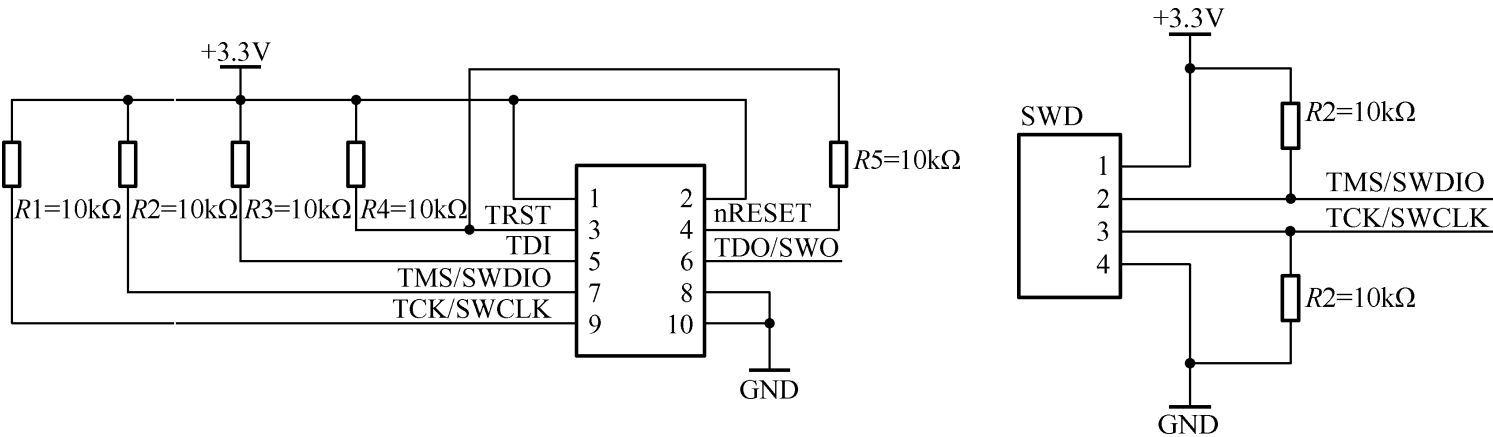
调试接口

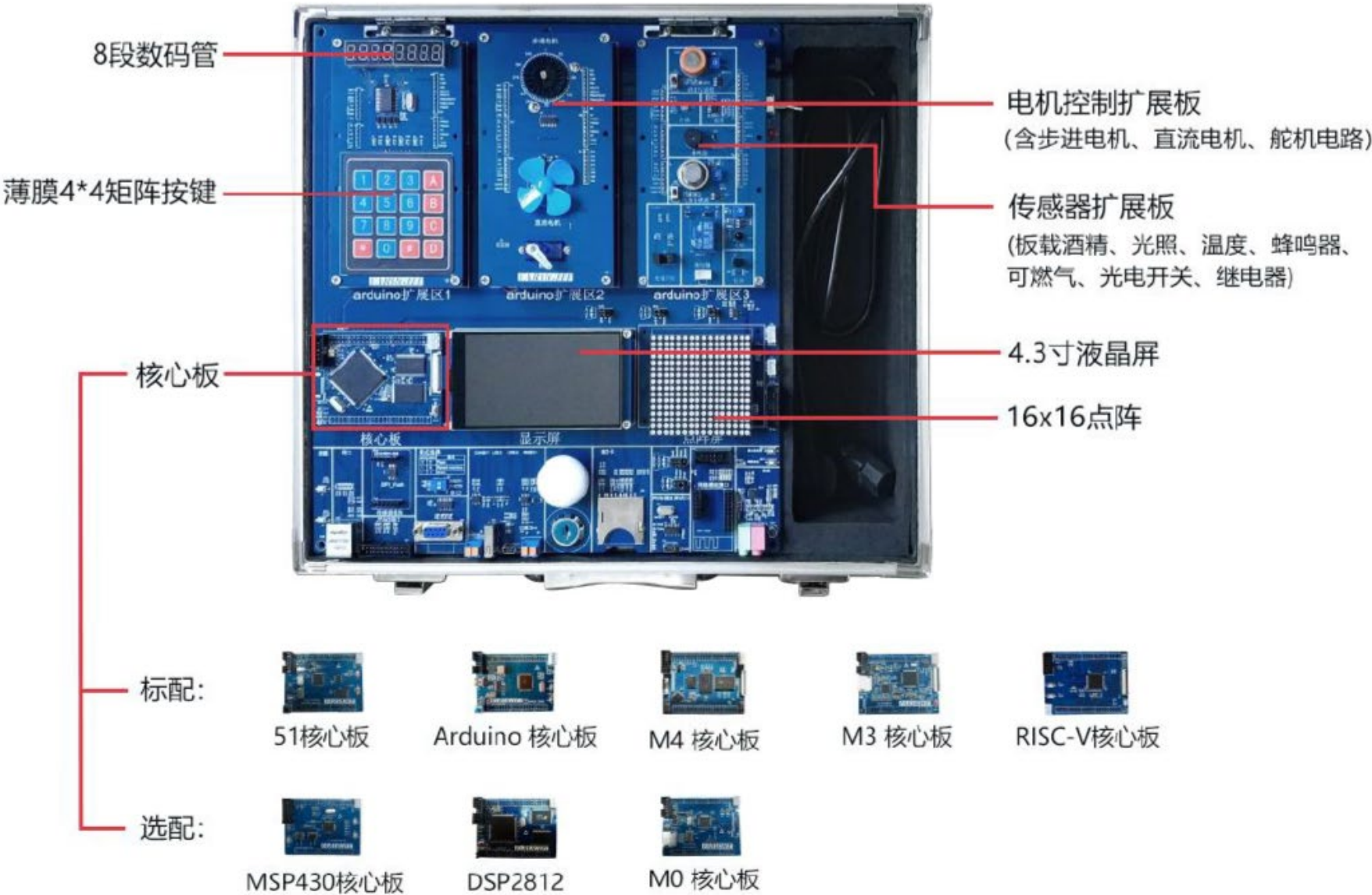
VCC	1	□	□	2	VCC	VCC	1	□	□	2	VCC
TRST	3	□	□	4	GND	N/C	3	□	□	4	GND
TDI	5	□	□	6	GND	N/C	5	□	□	6	GND
TMS	7	□	□	8	GND	SWDIO	7	□	□	8	GND
TCLK	9	□	□	10	GND	SWCLK	9	□	□	10	GND
RTCK	11	□	□	12	GND	N/C	11	□	□	12	GND
TDO	13	□	□	14	GND	SWO	13	□	□	14	GND
RESET	15	□	□	16	GND	RESET	15	□	□	16	GND
N/C	17	□	□	18	GND	N/C	17	□	□	18	GND
N/C	19	□	□	20	GND	N/C	19	□	□	20	GND

JTAG

SWD

STM32F系列微控制器调试接口引脚图





仿真器驱动安装

为了将开发环境编译生成的文件下载到芯片中并进行调试，需要调试下载工具将电脑和芯片进行连接。JLINK 和 STLINK 是两种常用的用于调试和编程ARM 微控制器的仿真器，它们都可以通过USB 接口与电脑连接，实现对目标芯片的内存、寄存器、外设等的访问和控制。ST-LINK 硬件主要由两部分组成：ST-LINK 主控板和连接线。ST-LINK 主控板上有一个USB 接口，一个20 针JTAG/SWD 连接口和LED 指示灯。USB 接口用于连接电脑，JTAG/SWD 连接口用于连接芯片进行调试和编程。各版本ST-LINK 如下图所示。



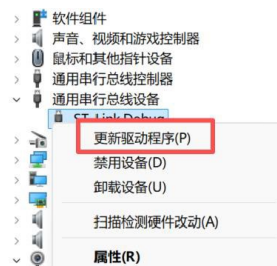
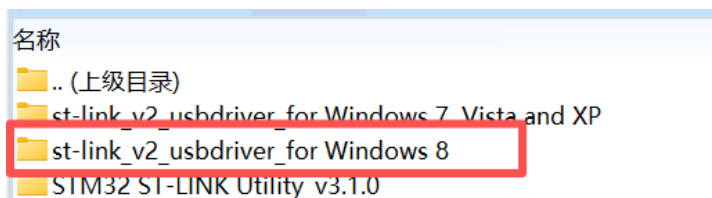
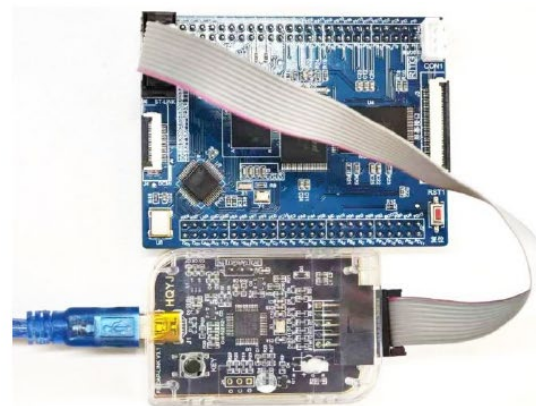
ST-LINK V1



ST-LINK V2



ST-LINK V3MINI



MDK 全称为Microcontroller Development Kit，即微控制器开发套件。它源自Keil 公司（后被ARM 公司收购），现也称为**MDK-ARM**。MDK-ARM 软件为基于Cortex-M、Cortex-R4、ARM7、ARM9 等内核的处理器设备提供了一个完整的开发环境。



armKEIL

Home / MDK

MDK Microcontroller Development Kit

Keil® MDK is the most comprehensive software development solution for Arm®-based microcontrollers and includes all components that you need to create, build, and debug embedded applications.

Download MDK

Update: MDK v5.36 contains Arm Compiler 6.16 and further enhancements.

Quick Links

Getting Started

Online Manuals

Compare Editions

Middleware

Learn

Software Packs

Device List

Evaluation Boards

MDK v4 Legacy Support

Third-Party Software Packs

Public Software Packs

Learning Platform

Product Components

MDK Tools

MDK-Core

μVision IDE & debugger with pack management

Arm C/C++ Compiler

With safety qualification

☐ 名称	修改日期
☐ 01.MDK官网下载地址.txt	2023/10/21 17:34
 Keil.STM32F1xx_DFP.2.0.0.pack	2023/10/21 17:05
 MDK-523.exe	2017/3/4 18:21

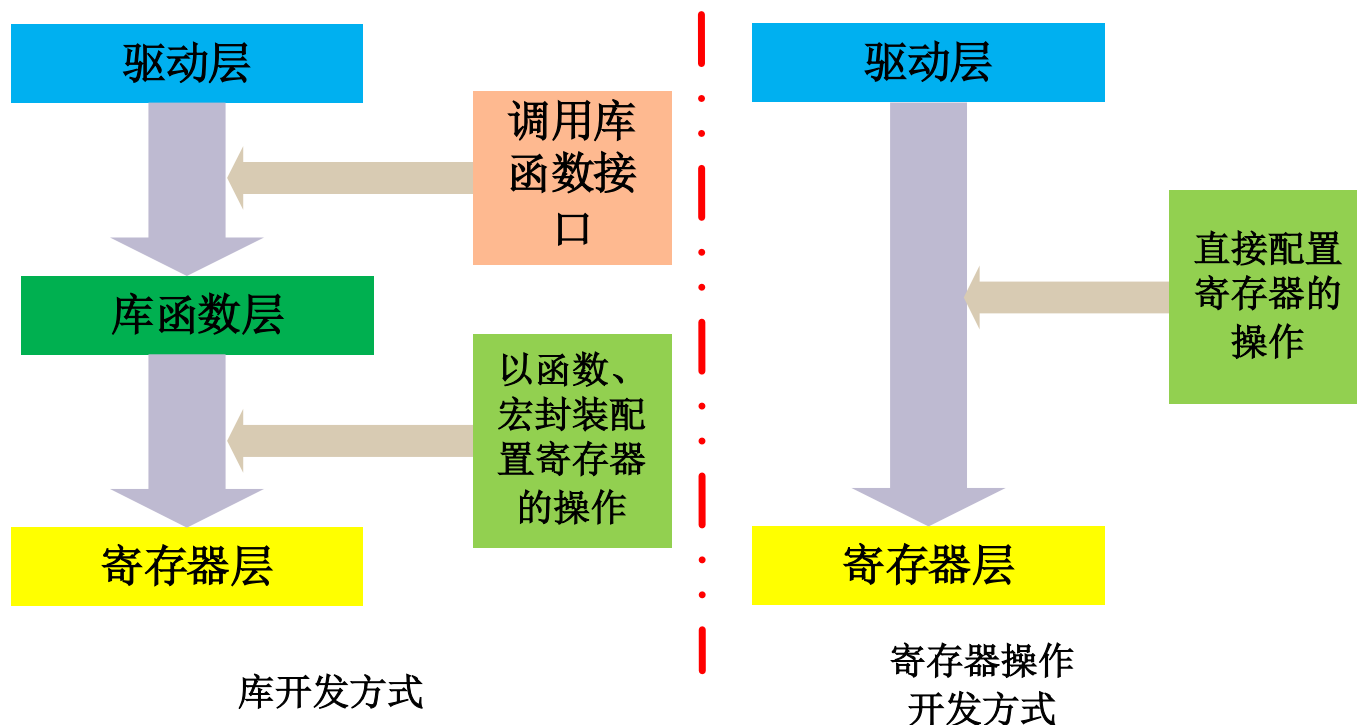
STM32CubeMX 是一种图形化的配置工具，通过分步过程可以非常轻松地配置STM32 微控制器和微处理器，以及为ARM Cortex-M 内核或面向ARM Cortex-A 内核的特定Linux 设备树生成相应的初始化C 代码。



 名称	修改日期	类型	大小
 01.JAVA官网下载地址.txt	2023/10/21 17:15	文本文档	1 KB
 02.STM32CubeMX官网下载地址.txt	2023/10/21 17:16	文本文档	1 KB
 03.MCU包官网下载地址.txt	2023/10/21 17:22	文本文档	1 KB
 jre-8u271-windows-x64.exe	2022/9/15 15:02	应用程序	81,411 KB
 SetupSTM32CubeMX-6.13.0-Win.exe	2024/11/22 17:36	应用程序	601,271 KB
 stm32cube_fw_f1_v160.zip	2023/12/19 14:12	压缩(zippped)文件夹	99,626 KB
 stm32cube_fw_f1_v180.zip	2023/10/21 17:42	压缩(zippped)文件夹	112,452 KB
 stm32cube_fw_f1_v186.zip	2025/2/24 17:42	压缩(zippped)文件夹	54,623 KB

软件开发形式

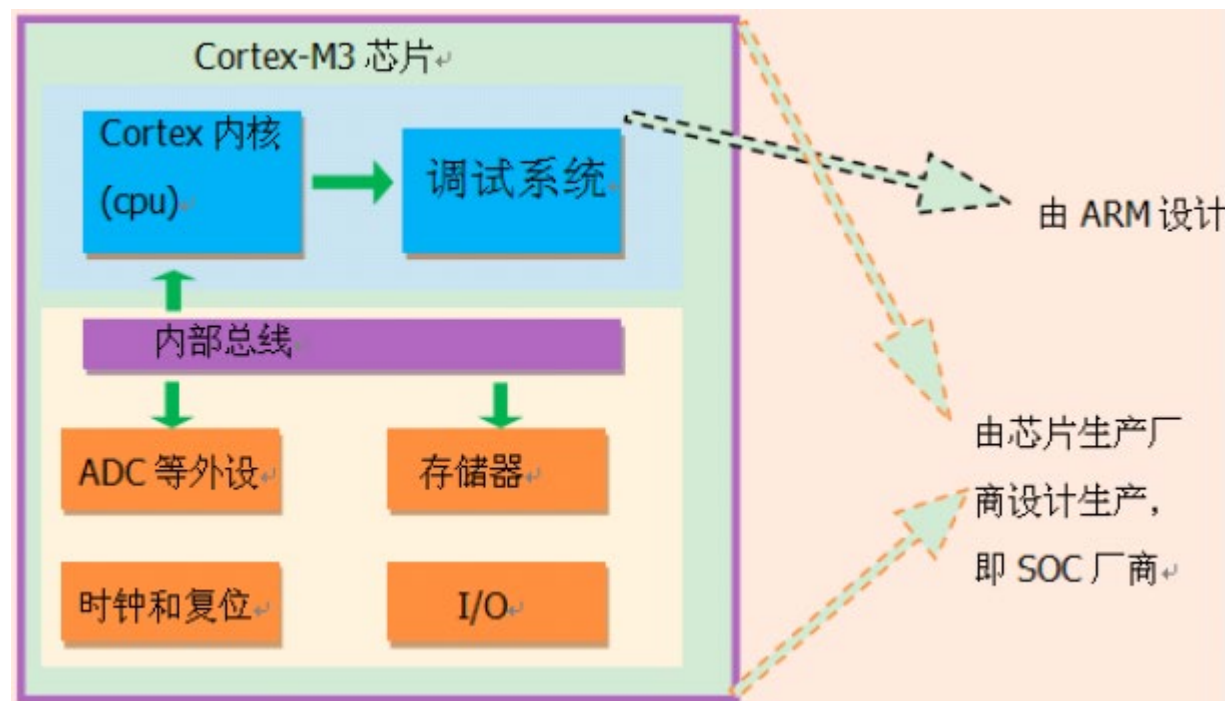
STM32库是由ST公司针对STM32提供的函数接口，即API (Application Program Interface)，开发者可调用这些函数接口来配置STM32的寄存器，使开发人员得以脱离最底层的寄存器操作，有开发快速，易于阅读，维护成本低等优点。



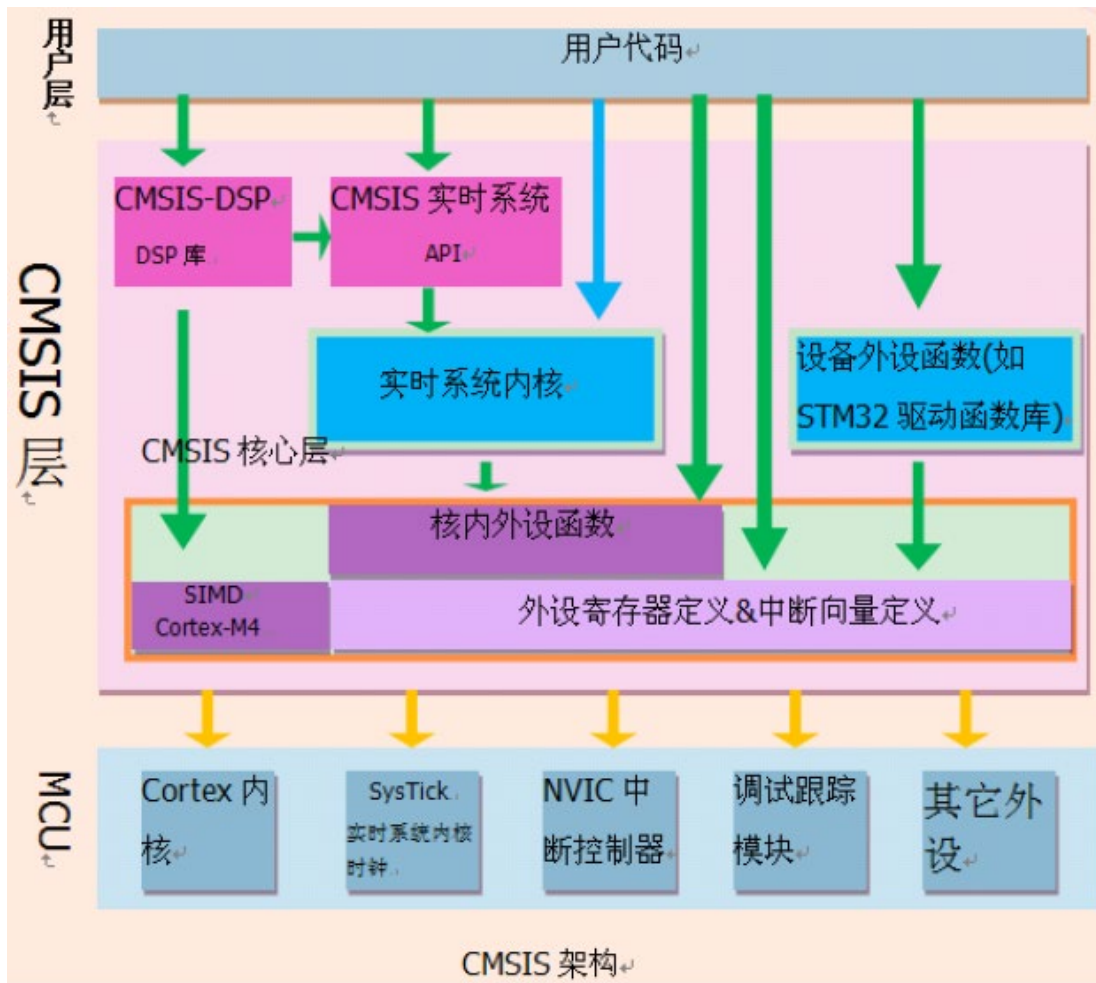
STM32标准函数库介绍

STM32的内核是ARM公司设计的处理器体系架构。ST公司或其它芯片生产厂商，负责设计的是在内核之外的部件，被称为核外外设或片上外设、设备外设。如芯片内部的模数转换外设ADC、串口UART、定时器TIM等。

为了解决不同的芯片厂商生产的Cortex微控制器软件的兼容性问题，ARM与芯片厂商建立了**CMSIS标准** (Cortex MicroController Software Interface Standard)。



CMSIS标准，实际是新建了一个软件抽象层



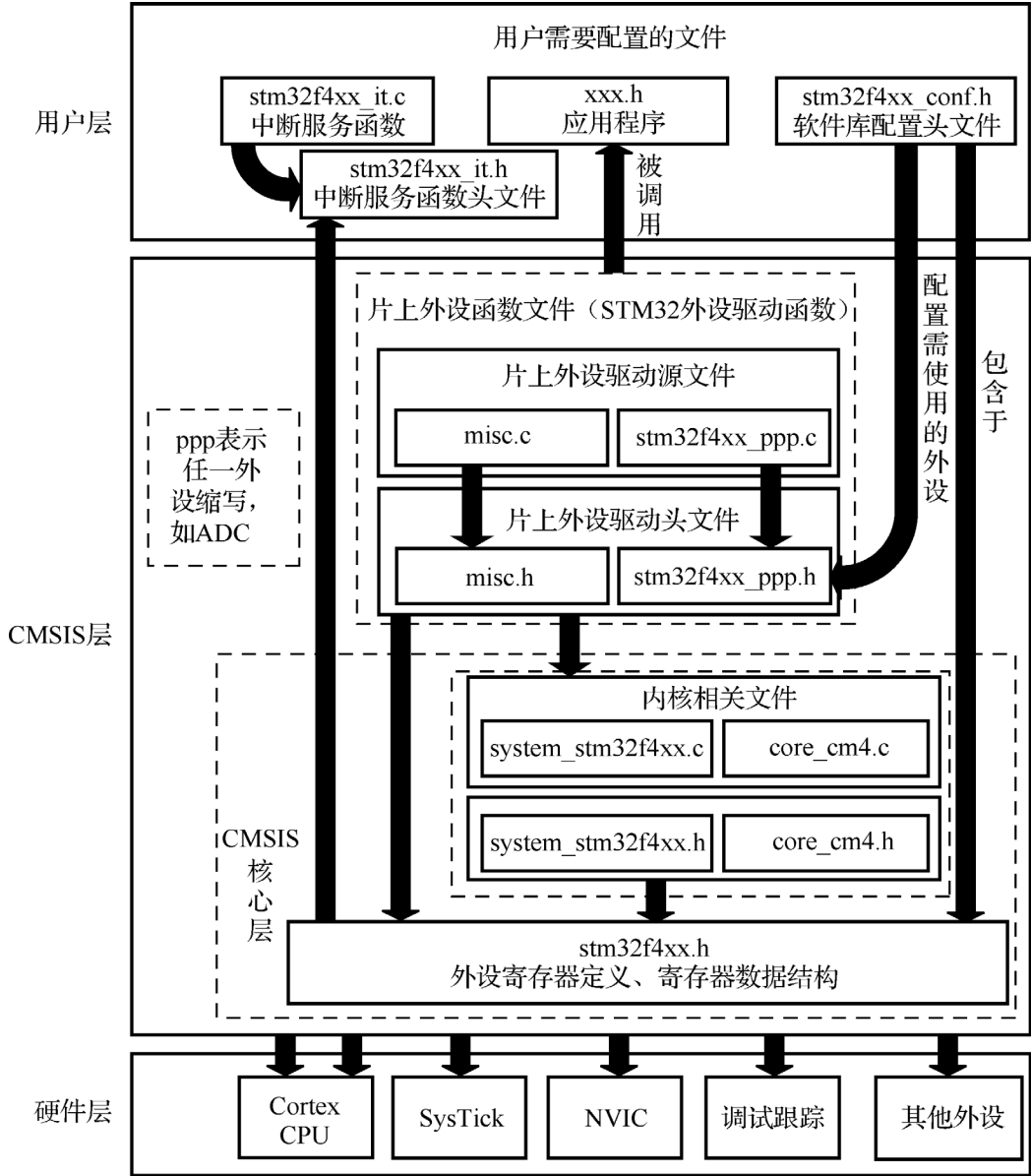
CMSIS标准中最主要的是CMSIS核心层，它包括了：

- 1、内核函数层：**其中包含用于访问内核寄存器的名称、地址定义，主要由ARM公司提供。
- 2、设备外设访问层：**提供了片上的核外外设的地址和中断定义，主要由芯片生产商提供。

CMSIS层位于**硬件层**与**操作系统或用户层**之间，提供了与芯片生产商无关的**硬件抽象层**，可以为接口外设、实时操作系统提供简单的处理器软件接口，屏蔽了硬件差异，这对软件的移植是有极大的好处的。

STM32的库，就是按照CMSIS标准建立的。

STM32标准函数库介绍



STM32标准函数库介绍

STM32F4xx标准函数库文件分析			
启动文件	startup_stm32f429_439xx.s	启动文件	1 必须
外设相关	stm32f4xx.h	外设寄存器定义	
	system_stm32f4xx.h	用于系统初始化	
	system_stm32f4xx.c	用于配置系统时钟	2 必须
	stm32f4xx_xx.h	外设标准函数库头文件	
	stm32f4xx_xx.c	外设标准函数库源件	3 必须，与外设相关
	misc.h		
	misc.c	NVIC、SysTick相关函数	4 必须
内核相关	core_cm4.h	内核寄存器定义	
	core_cmFunc.h	操作内核相关，不常用	
	core_cmInstr.h	定义了内核指令	
	core_cmSimd.h	定义SIMD指令	
用户相关	stm32f4xx_it.h		
	stm32f4xx_it.c	用户编写的中断服务函数	5
	main.c	用户应用程序主程序入口	6 必须
	其他应用子程序	用户自定义应用功能	7

STM32标准函数库介绍

标准函数库使用

需要使用哪一个片上外设即将库中，其对应的源文件添加到工程中。

同时，在stm32f4xx_conf.h包含对应头文件。

GPIO相关的操作函数

源文件

stm32f4xx_XX.c

头文件

- misc.c
- stm32f4xx_adc.c
- stm32f4xx_can.c
- stm32f4xx_cec.c
- stm32f4xx_crc.c
- stm32f4xx_cryp.c
- stm32f4xx_cryp_aes.c
- stm32f4xx_cryp_des.c
- stm32f4xx_cryp_tdes.c
- stm32f4xx_dac.c
- stm32f4xx_dbgmcu.c
- stm32f4xx_dcmi.c
- stm32f4xx_dfsdm.c
- stm32f4xx_dma.c
- stm32f4xx_dma2d.c
- stm32f4xx_dsi.c
- stm32f4xx_exti.c
- stm32f4xx_flash.c
- stm32f4xx_flash_ramfunc.c
- stm32f4xx_fmc.c
- stm32f4xx_fmpi2c.c
- stm32f4xx_fsmc.c
- stm32f4xx_gpio.c
- stm32f4xx_hash.c
- stm32f4xx_hash_md5.c

- misc.h
- stm32f4xx_adc.h
- stm32f4xx_can.h
- stm32f4xx_cec.h
- stm32f4xx_crc.h
- stm32f4xx_cryp.h
- stm32f4xx_dac.h
- stm32f4xx_dbgmcu.h
- stm32f4xx_dcmi.h
- stm32f4xx_dfsdm.h
- stm32f4xx_dma.h
- stm32f4xx_dma2d.h
- stm32f4xx_dsi.h
- stm32f4xx_exti.h
- stm32f4xx_flash.h
- stm32f4xx_flash_ramfunc.h
- stm32f4xx_fmc.h
- stm32f4xx_fmpi2c.h
- stm32f4xx_fsmc.h
- stm32f4xx_gpio.h
- stm32f4xx_hash.h
- stm32f4xx_j2c.h
- stm32f4xx_iwdg.h
- stm32f4xx_lptim.h
- stm32f4xx_ltdc.h

STM32标准函数库介绍

标准函数库使用

1、startup_stm32f429_439xx.s

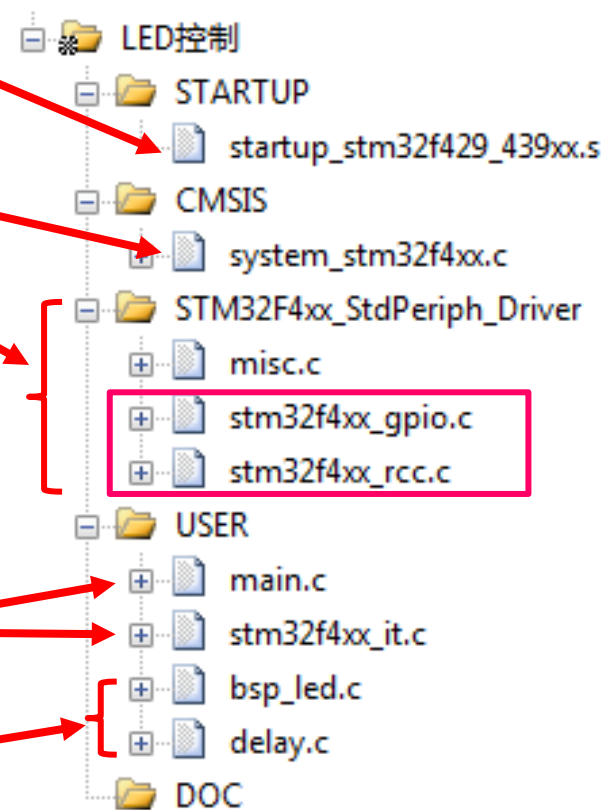
2、system_stm32f4xx.c

3、stm32f4xx_xx.c：LED控制需要使用到GPIO，并用到了系统时钟定时生成的延时功能，因此在工程中需要包含需要包含stm32f4xx_gpio.c和stm32f4xx_rcc.c

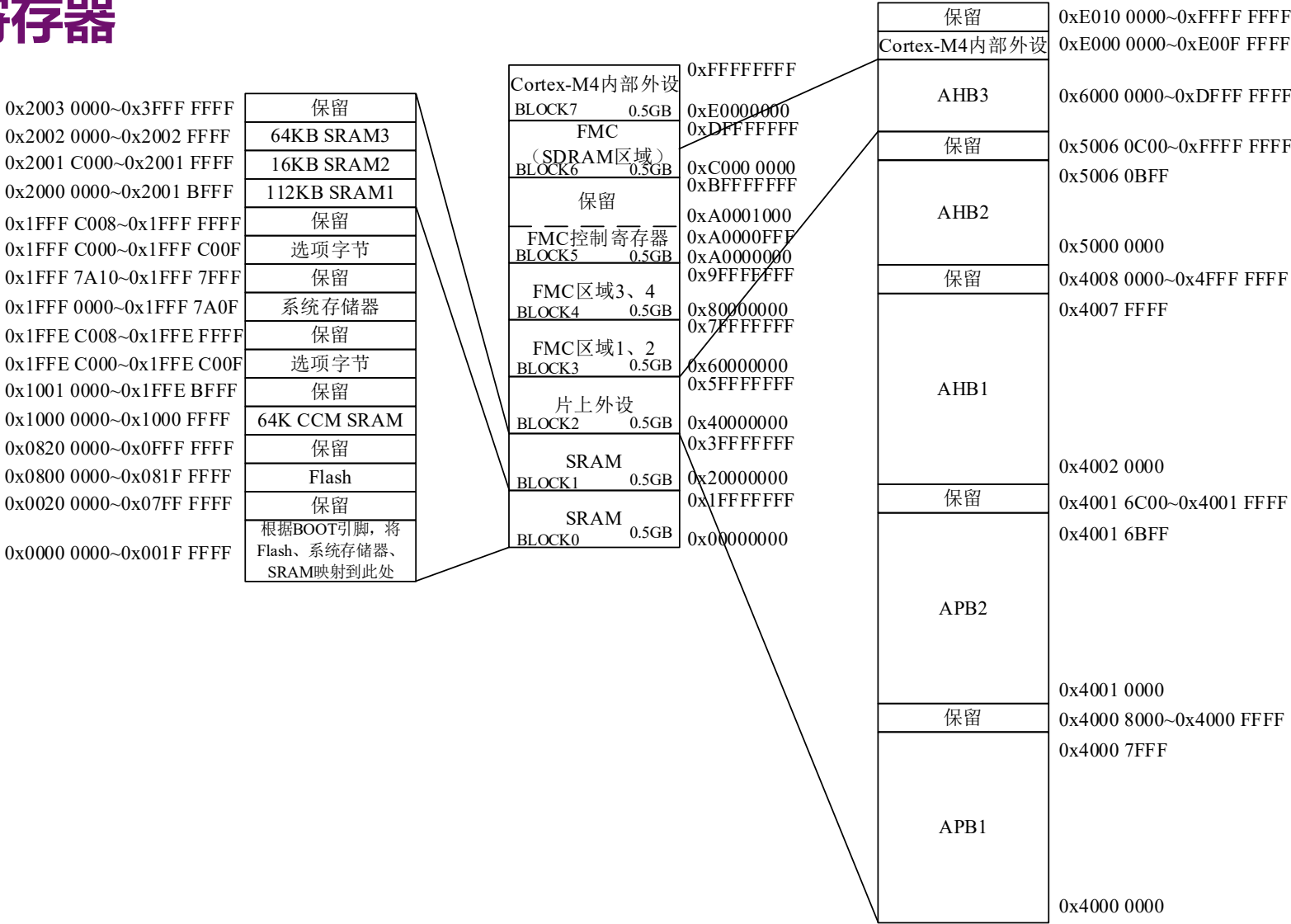
4、stm32f4xx_it.c

5、main.c

6、其他应用程序



存储器映射和寄存器



存储器映射和寄存器

把片上外设对应的寄存器在存储空间上分配地址的过程就叫**寄存器映射**。
与存储单元一样，每个寄存器（一般都是32位的）都有一个寻址地址。

寄存器是一个很重要的概念。

- ◆ 应用程序对片上外设的初始化和控制是**通过对片上外设对应一系列寄存器的修改、读写来实现的。**

存储器映射和寄存器

GPIOH端口的寄存器列表

寄存器名称	寄存器地址	相对GPIOH基址的偏移
GPIOH_MODER	0x4002 1C00	0x00
GPIOH_OTYPER	0x4002 1C04	0x04
GPIOH_OSPEEDR	0x4002 1C08	0x08
GPIOH_PUPDR	0x4002 1C0C	0x0C
GPIOH_IDR	0x4002 1C10	0x10
GPIOH_ODR	0x4002 1C14	0x14
GPIOH_BSRR	0x4002 1C18	0x18
GPIOH_LCKR	0x4002 1C1C	0x1C
GPIOH_AFRL	0x4002 1C20	0x20
GPIOH_AFRH	0x4002 1C24	0x24

存储器映射和寄存器

GPIOx端口数据输出寄存器ODR描述

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	③
Reserved																②
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	③
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0	②
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	①

位31:16 保留，必须保持复位值。

位15:0 ODRy[15:0]：端口输出数据（y=0~15）这些位可通过软件读取和写入。

}

④

①是寄存器中位段的操作权限，r表示只读，w表示只写，rw表示可读可写。②是位段名。③是位段编号，从0开始。④是对寄存器各位段的使用说明，通过这一部分可以得到这一个寄存器所能实现的功能。

端口有16个引脚，编号对应0-15。数据输出寄存器的低16为对应于端口的没一个引脚。

存储器映射和寄存器

让GPIOH端口的16个引脚输出高电平

通过绝对地址访问内存单元

```
1 // GPIOH 端口全部输出 高电平
2 *(unsigned int*)(0x40021C14) = 0xFFFF;
```

问题:

- 1) 、 0X40021C14 是GPIOH输出数据寄存器ODR的地址, 如何找到?
- 2) 、 (unsigned int*)的作用是什么?
- 2) 、 学会使用C语言的 * 号

存储器映射和寄存器

通过宏定义方式访问内存单元

```
1 // GPIOH 端口全部输出 高电平
2 #define GPIOH_ODR      (unsignedint*) (0x40021C14)
3 * GPIOH_ODR = 0xFFFF;
```

为了方便操作，我们干脆把指针操作 “*” 也定义到寄存器别名里面

```
1 // GPIOH 端口全部输出 高电平
2 #define GPIOH_ODR      *(unsignedint*) (0x40021C14)
3 GPIOH_ODR = 0xFFFF;
```

存储器映射和寄存器

1、总线和外设基址宏定义

定义在头文件
STM32F4xx.h中

```
/* 外设基址 */
#define PERIPH_BASE                ((unsigned int)0x40000000)
/* 总线基址 */
#define APB1PERIPH_BASE            PERIPH_BASE
#define APB2PERIPH_BASE            (PERIPH_BASE + 0x00010000)
#define AHB1PERIPH_BASE            (PERIPH_BASE + 0x00020000)
#define AHB2PERIPH_BASE            (PERIPH_BASE + 0x10000000)
/* GPIO外设基址 */
#define GPIOA_BASE                  (AHB1PERIPH_BASE + 0x0000)
#define GPIOB_BASE                  (AHB1PERIPH_BASE + 0x0400)
#define GPIOC_BASE                  (AHB1PERIPH_BASE + 0x0800)
#define GPIOD_BASE                  (AHB1PERIPH_BASE + 0x0C00)
#define GPIOE_BASE                  (AHB1PERIPH_BASE + 0x1000)
#define GPIOF_BASE                  (AHB1PERIPH_BASE + 0x1400)
#define GPIOG_BASE                  (AHB1PERIPH_BASE + 0x1800)
#define GPIOH_BASE                  (AHB1PERIPH_BASE + 0x1C00)
/*使用把地址强制转换成GPIO_TypeDef类型地址*/
#define GPIOA                      ((GPIO_TypeDef *) GPIOA_BASE)
#define GPIOB                      ((GPIO_TypeDef *) GPIOB_BASE)
#define GPIOC                      ((GPIO_TypeDef *) GPIOC_BASE)
#define GPIOD                      ((GPIO_TypeDef *) GPIOD_BASE)
#define GPIOE                      ((GPIO_TypeDef *) GPIOE_BASE)
#define GPIOF                      ((GPIO_TypeDef *) GPIOF_BASE)
#define GPIOG                      ((GPIO_TypeDef *) GPIOG_BASE)
#define GPIOH                      ((GPIO_TypeDef *) GPIOH_BASE)
```


存储器映射和寄存器

2、使用结构体封装寄存器

定义在头文件
STM32F4xx.h中

```
typedef unsigned int uint32_t; /*无符号32位变量*/
typedef unsigned short int uint16_t; /*无符号16位变量*/

/* GPIO寄存器列表 */
typedef struct {
    uint32_t MODER; /*GPIO模式寄存器 地址偏移: 0x00 */
    uint32_t OTYPER; /*GPIO输出类型寄存器 地址偏移: 0x04 */
    uint32_t OSPEEDR; /*GPIO输出速度寄存器 地址偏移: 0x08 */
    uint32_t PUPDR; /*GPIO上拉/下拉寄存器 地址偏移: 0x0C */
    uint32_t IDR; /*GPIO输入数据寄存器 地址偏移: 0x10 */
    uint32_t ODR; /*GPIO输出数据寄存器 地址偏移: 0x14 */
    uint16_t BSRRL; /*GPIO置位/复位寄存器低16位部分 地址偏移: 0x18 */
    uint16_t BSRRH; /*GPIO置位/复位寄存器高16位部分 地址偏移: 0x1A */
    uint32_t LCKR; /*GPIO配置锁定寄存器 地址偏移: 0x1C */
    uint32_t AFR[2]; /*GPIO复用功能配置寄存器 地址偏移: 0x20-0x24 */
} GPIO_TypeDef;
```

注意：寄存器封装形式可以自定义！

存储器映射和寄存器

例：将GPIOH端口的10号引脚先输出低电平，再输出高电平。

```
// PH10输出输出低电平  
GPIOH->ODR  &= ~(1<<10);
```

```
// PH10输出输出高电平  
GPIOH->ODR  |= (1<<10);
```

注意：

1) 在对寄存器操作的时候一般通过读-修改-写的方式实现。

常用操作有：**位与&（清零），位或|（置位），异或^（取反）**；

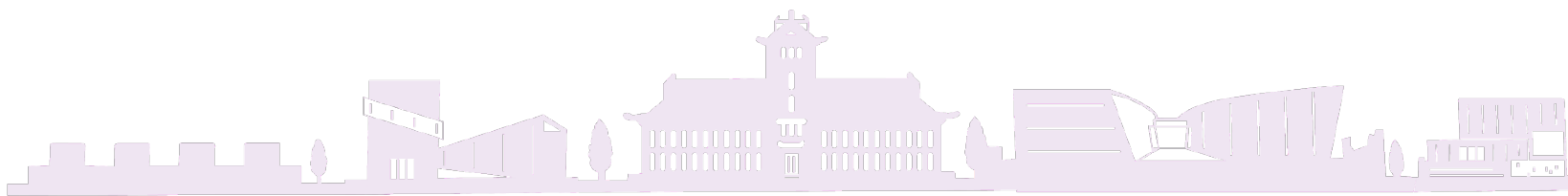
2) 选择合适的操作和对应的屏蔽字。

本章小结

01 STM32概述

02 最小系统设计

03 开发环境搭建



谢谢!